Dot Net Technology

Elective

Introduction to C# Language

C# is a general-purpose, object-oriented programming language. **C# was developed by Anders Hejlsberg and his team during the development of .Net Framework.** It can be used to create desktop, web and mobile applications.

C# is a hybrid of C and C++; it is a Microsoft programming language developed to compete with Sun's Java language. C# is an object-oriented programming language used with XML- based Web services on the .NET platform and designed for improving productivity in the development of Web applications.

C# is an elegant and type-safe object-oriented language that enables developers to build a variety of secure and robust applications that run on the .NET Framework. You can use C# to create Windows client applications, XML Web services, distributed components, client- server applications, database applications, and much, much more. Visual C# provides an advanced code editor, convenient user interface designers, integrated debugger, and many other tools to make it easier to develop applications based on the C# language and the .NET Framework.

As an object-oriented language, C# supports the concepts of encapsulation, inheritance, and polymorphism. All variables and methods, including the Main method, the application's entry point, are encapsulated within class definitions.

The following reasons make C# a widely used professional language (Features of C#):

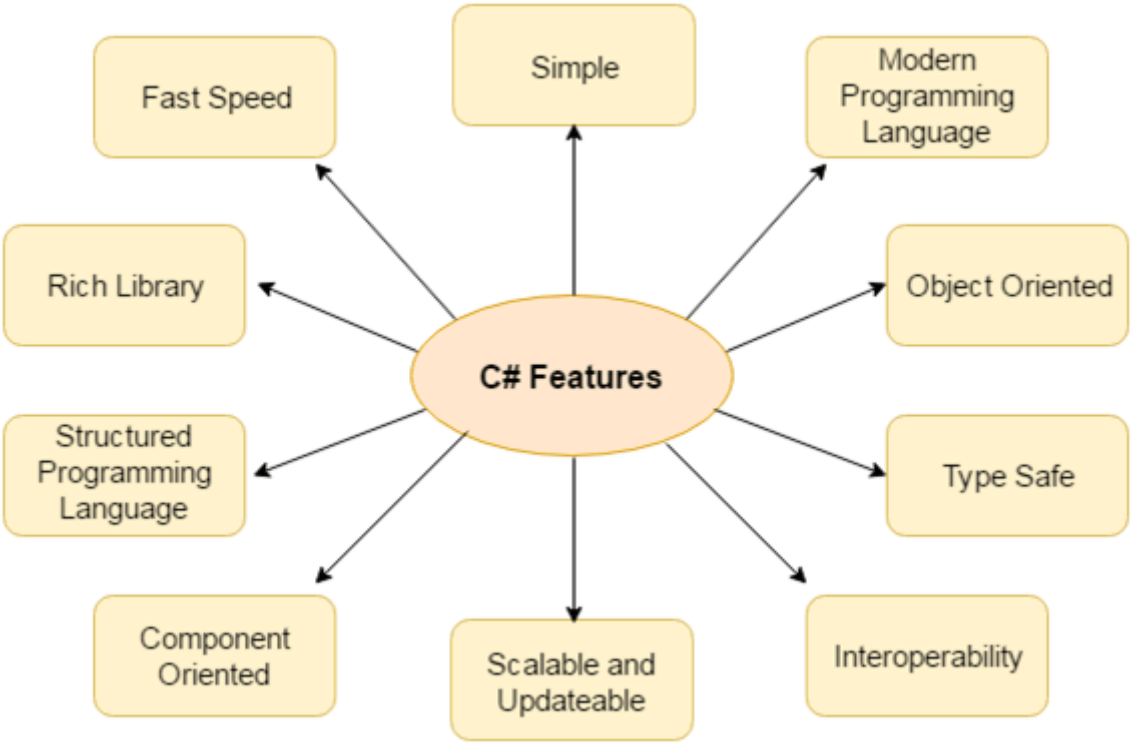
1. It is a modern, general-purpose programming language
2. It is object oriented.
3. It is component oriented.
4. It is easy to learn.
5. It is a structured language.
6. It produces efficient programs.
7. It can be compiled on a variety of computer platforms.
8. It is a part of .Net Framework.

C# Features

C# is object-oriented programming language. It provides a lot of **features** that are given below.

1. Simple
2. Modern programming language

3. Object oriented
4. Type safe
5. Interoperability
6. Scalable and Updateable
7. Component oriented
8. Structured programming language
9. Rich Library
10. Fast speed



1) Simple

C# is a simple language in the sense that it provides structured approach (to break the problem into parts), rich set of library functions, data types etc.

2) Modern Programming Language

C# programming is based upon the current trend and it is very powerful and simple for building scalable, interoperable and robust applications.

3) Object Oriented

C# is object-oriented programming language. OOPs makes development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grow.

4) Type Safe

C# type safe code can only access the memory location that it has permission to execute. Therefore it improves a security of the program.

5) Interoperability

Interoperability process enables the C# programs to do almost anything that a native C++ application can do.

6) Scalable and Updateable

C# is automatic scalable and updateable programming language. For updating our application we delete the old files and update them with new ones.

7) Component Oriented

C# is component oriented programming language. It is the predominant software development methodology used to develop more robust and highly scalable applications.

8) Structured Programming Language

C# is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify.

9) Rich Library

C# provides a lot of inbuilt functions that makes the development fast.

10) Fast Speed

The compilation and execution time of C# language is fast.

Object Orientation

C# is a rich implementation of the object-orientation paradigm, which includes encapsulation, abstraction, inheritance, and polymorphism. Encapsulation means creating a boundary around an object, to separate its external (public) behavior from its internal (private) implementation details.

The distinctive features of C# from an object-oriented perspective are:

Unified type system

The fundamental building block in C# is an encapsulated unit of data and functions called a type. C#

has a unified type system, where all types ultimately share a common base type. This means that all types, whether they represent business objects or are primitive types such as numbers, share the same basic functionality. For example, an instance of any type can be converted to a string by calling its ToString method.

Classes and interfaces

In a traditional object-oriented paradigm, the only kind of type is a class. In C#, there are several other kinds of types, one of which is an interface. An interface is like a class, except that it only describes members. The implementation for those members comes from types that implement the interface. Interfaces are particularly useful in scenarios where multiple inheritance is required (unlike languages such as C++ and Eiffel, C# does not support multiple inheritance of classes).

Properties, methods, and events

In the pure object-oriented paradigm, all functions are methods. In C#, methods are only one kind of function member, which also includes properties and events. Properties are function members that encapsulate a piece of an object's state, such as a button's color or a label's text. Events are function members that simplify acting on object state changes.

While C# is primarily an object-oriented language, it also borrows from the functional programming paradigm. Specifically:

Functions can be treated as values

Through the use of delegates, C# allows functions to be passed as values to and from other functions.

C# supports patterns for purity

Core to functional programming is avoiding the use of variables whose values change, in favour of declarative patterns. C# has key features to help with those patterns, including the ability to write unnamed functions on the fly that "capture" variables (lambda expressions), and the ability to perform list or reactive programming via query expressions. C# also makes it easy to define read-only fields and properties for writing immutable (read- only) types.

Type Safety

C# is primarily a type-safe language, meaning that instances of types can interact only through protocols they define, thereby ensuring each type's internal consistency. For instance, C# prevents you from interacting with a string type as though it were an integer type.

More specifically, C# supports static typing, meaning that the language enforces type safety at compile time. This is in addition to type safety being enforced at run-time.

Static typing eliminates a large class of errors before a program is even run. It shifts the burden away from runtime unit tests onto the compiler to verify that all the types in a program fit together correctly. This makes large programs much easier to manage, more predictable, and more robust.

Furthermore, static typing allows tools such as IntelliSense in Visual Studio to help you write a program, since it knows for a given variable what type it is, and hence what methods you can call on that variable.

C# is also called a strongly typed language because its type rules (whether enforced statically or at runtime) are very strict. For instance, you cannot call a function that's designed to accept an integer with a floating-point number, unless you first explicitly convert the floating-point number to an integer. This helps prevent mistakes.

Strong typing also plays a role in enabling C# code to run in a sandbox—an environment where every aspect of security is controlled by the host. In a sandbox, it is important that you cannot arbitrarily corrupt the state of an object by bypassing its type rules.

Memory Management

C# relies on the runtime to perform automatic memory management. The Common Language Runtime has a garbage collector that executes as part of your program, reclaiming memory for objects that are no longer referenced. This frees programmers from explicitly locating the memory for an object, eliminating the problem of incorrect pointers encountered in languages such as C++.

C# does not eliminate pointers: it merely makes them unnecessary for most programming tasks.

Platform Support

Historically, C# was used almost entirely for writing code to run on Windows platforms. Recently, however, Microsoft and other companies have invested in other platforms, including Linux, macOS, iOS, and Android.

Xamarin™ allows cross platform C# development for mobile applications, and Portable Class Libraries are becoming increasingly widespread.

Microsoft's ASP.NET Core is a cross-platform lightweight web hosting framework that can run either on the .NET Framework or on .NET Core, an open source cross-platform runtime.

C# and CLR

C# depends on a runtime equipped with a host of features such as automatic memory management and exception handling. **At the core of the Microsoft .NET Framework is the Common Language Runtime (CLR), which provides these runtime features**. (The .NET Core and Xamarin frameworks provide similar runtimes.)

The CLR is language-neutral, allowing developers to build applications in multiple languages (e.g., C#, F#, Visual Basic .NET, and Managed C++).

C# is one of several managed languages that get compiled into managed code. Man- aged code is

represented in Intermediate Language or IL. The CLR converts the IL into the native code of the machine, such as X86 or X64, usually just prior to execution. This is referred to as Just-In-Time (JIT) compilation. Ahead-of-time compilation is also available to improve start up time with large assemblies or resource constrained devices (and to satisfy iOS app store rules when developing with Xamarin).

The container for managed code is called an assembly or portable executable. An assembly can be an executable file (.exe) or a library (.dll), and contains not only IL, but type information (metadata). The presence of metadata allows assemblies to reference types in other assemblies without needing additional files.

A program can query its own metadata (reflection), and even generate new IL at runtime (reflection.emit)

Introduction and Overview of .Net Framework

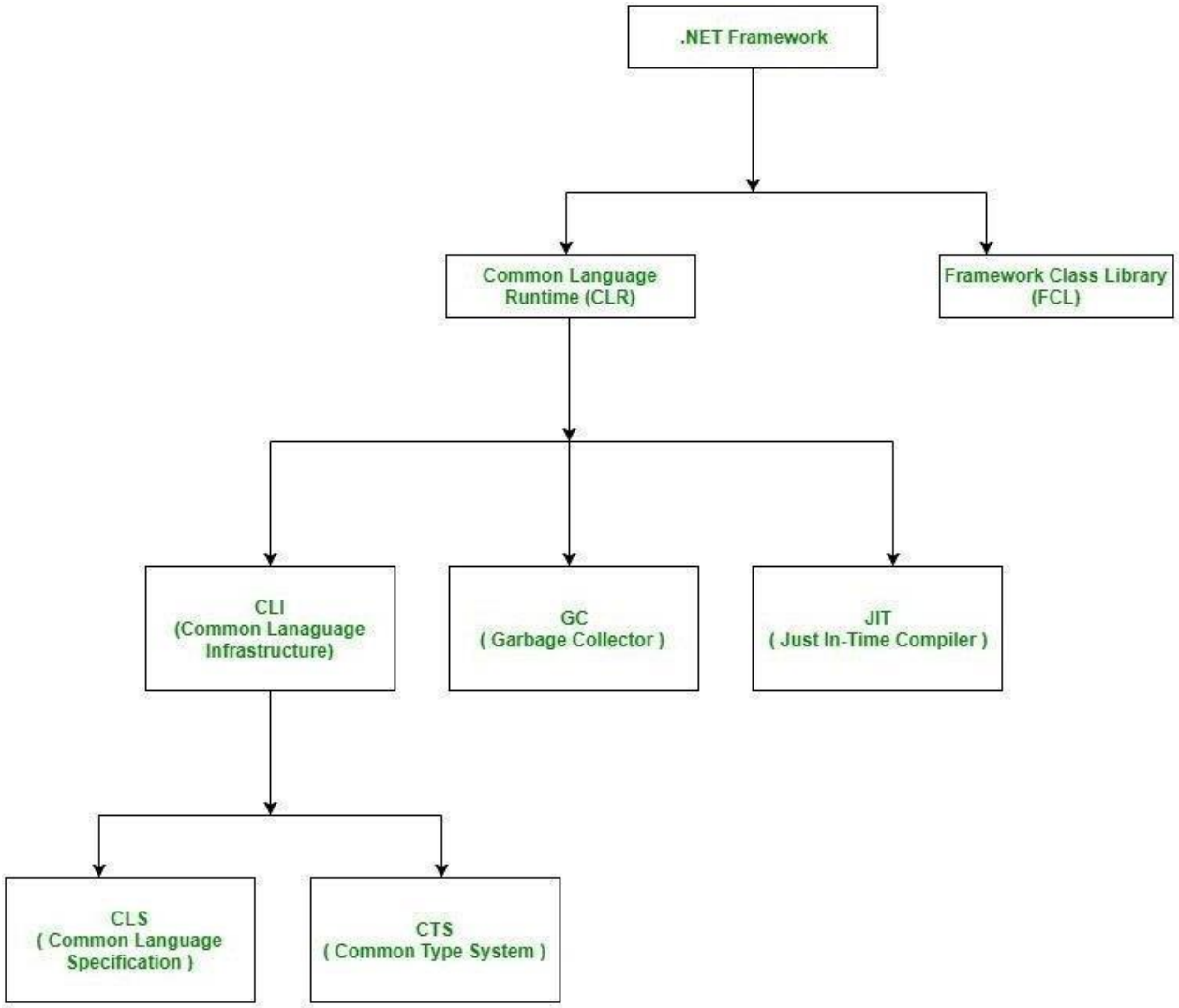
.NET is the framework for which we develop applications. It sits in between our application programs and operating system.

.NET provides an object oriented environment. It ensures safe execution of the code by performing required runtime validations. For example, it is never possible to access an element of an array outside the boundary. Similarly, it is not possible to a program to write into another programs area, etc. The runtime validations performed by .NET makes the entire environment robust.

It is a programming infrastructure created by Microsoft for building, deploying, and running applications and services that use .NET technologies, such as desktop applications and Web services.

The .NET Framework consists of:

- the Common Language Runtime
- the Framework Class Library



Common Language Runtime(CLR):

CLR is the basic and Virtual Machine component of the .NET Framework. It is the run-time environment in the .NET Framework that runs the codes and helps in making the development process easier by providing the various services such as remoting, thread management, type-safety, memory management, robustness etc. Basically, it is responsible for managing the execution of .NET programs regardless of any .NET programming language. It also helps in the management of code, as code that targets the runtime is known as the Managed Code and code doesn't target to runtime is known as Unmanaged code.

Features of Common Language Runtime:

1. The runtime enforces code access security. For example, users can trust that an executable

embedded in a Web page can play an animation on screen or sing a song, but cannot access their personal data, file system, or network.

2. The runtime also enforces code robustness by implementing a strict type-and-code-verification infrastructure called the common type system (CTS). The CTS ensures that all managed code is self-describing.

3. The runtime also accelerates developer productivity. For example, programmers can write applications in their development language of choice, yet take full advantage of the runtime, the class library, and components written in other languages by other developers. Any compiler vendor who chooses to target the runtime can do so. Language compilers that target the .NET Framework make the features of the .NET Framework available to existing code written in that language, greatly easing the migration process for existing applications.

4. While the runtime is designed for the software of the future, it also supports software of today and yesterday.

5. The runtime is designed to enhance performance.

6. The runtime can be hosted by high-performance, server-side applications, such as Microsoft SQL Server and Internet Information Services (IIS).

Framework Class Library (FCL):

The class library is a comprehensive, object-oriented collection of reusable types that you can use to develop applications ranging from traditional command-line or graphical user interface (GUI) applications to applications based on the latest innovations provided by ASP.NET, such as Web Forms and XML Web services.

It is the collection of reusable, object-oriented class libraries and methods etc. that can be integrated with CLR. Also called the Assemblies. It is just like the header files in C/C++ and packages in the java. Installing .NET framework basically is the installation of CLR and FCL into the system.

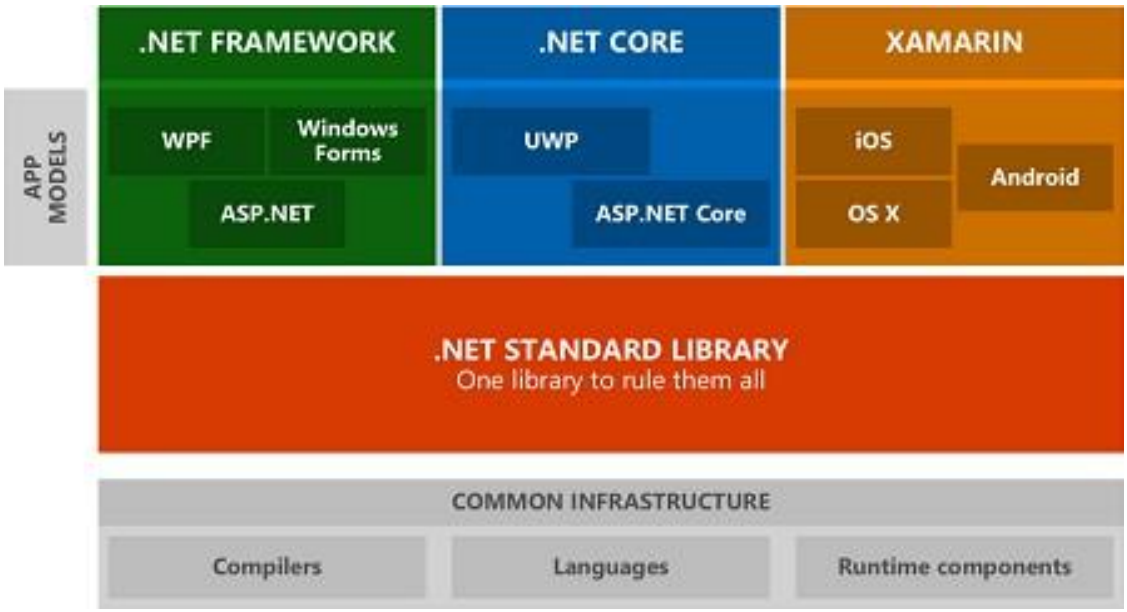
The core libraries are sometimes collectively called the Base Class Library (BCL).
The entire framework is called the Framework Class Library (FCL).

Features of Framework Class Library:

An object-oriented class library, the .NET Framework types enable you to accomplish a range of common programming tasks, including tasks such as string management, data collection, database connectivity, and file access. In addition to these common tasks, the class library includes types that support a variety of specialized development scenarios. For example, you can use the .NET Framework to develop the following types of applications and services:

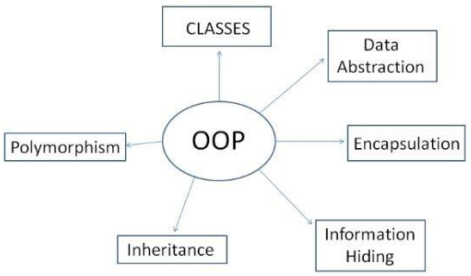
1. Console applications

2. Windows GUI applications (Windows Forms).
3. Windows Presentation Foundation (WPF) applications.
4. ASP.NET applications.
5. Windows services.
6. Service-oriented applications using Windows Communication Foundation (WCF).
7. Workflow-enabled applications using Windows Workflow Foundation (WF).



Feature of Object Oriented Programming:

Object-oriented programming (OOP) refers to a type of computer programming (software design) in which programmers define not only the data type of a data structure, but also the types of operations (functions) that can be applied to the data structure.



In this way, the data structure becomes an object that includes both data and functions. In addition, programmers can create relationships between one object and another. For example, objects can inherit characteristics from other objects.

Following are the features of OOPS:

Abstraction: The process of picking out (abstracting) common features of objects and procedures.

Class: A category of objects. The class defines all the common properties of the different objects that belong to it.

Encapsulation: The process of combining elements to create a new entity. A procedure is a type of encapsulation because it combines a series of computer instructions.

Information hiding: The process of hiding details of an object or function. Information hiding is a powerful programming technique because it reduces complexity.

Inheritance: a feature that represents the "is a" relationship between different classes. **Interface:** the languages and codes that the applications use to communicate with each other and with the hardware.

Object: a self-contained entity that consists of both data and procedures to manipulate the data.

Polymorphism: A programming language's ability to process objects differently depending on their data type or class.

Procedure: a section of a program that performs a specific task.

Procedure-Oriented vs. Object-Oriented Programming

Object-Oriented Programming (OOP) is a high-level programming language where a program is divided into small chunks called objects using the object-oriented model, hence the name. This paradigm is based on objects and classes.

- **Object –** An object is basically a self-contained entity that accumulates both data and procedures to manipulate the data. Objects are merely instances of classes.
- **Class –** A class, in simple terms, is a blueprint of an object which defines all the common properties of one or more objects that are associated with it. A class can be used to define multiple objects within a program.

The OOP paradigm mainly eyes on the data rather than the algorithm to create modules by dividing a program into data and functions that are bundled within the objects. The modules cannot be modified when a new object is added restricting any non-member function access to the data. Methods are the only way to assess the data.

Objects can communicate with each other through same member functions. This process is known as message passing. This anonymity among the objects is what makes the program secure. A programmer can create a new object from the already existing objects by taking most of its features thus making the program easy to implement and modify.

Procedure-Oriented Programming (POP) follows a step-by-step approach to break down a task into a collection of variables and routines (or subroutines) through a sequence of instructions. Each step is carried out in order in a systematic manner so that a computer can understand what to do. The program is divided into small parts called functions and then it follows a series of computational steps to be carried out in order.

It follows a top-down approach to actually solve a problem, hence the name. Procedures correspond to functions and each function has its own purpose. Dividing the program into functions is the key to procedural programming. So a number of different functions are written in order to accomplish the tasks.

Difference between OOP and POP is shown below:

Procedure Oriented Programming	**Object Oriented Programming**
In POP, program is divided into small parts called **functions**.	In OOP, program is divided into parts called **objects**.
In POP, Importance is not given to **data** but to functions as well as **sequence** of actions to be done.	In OOP, Importance is given to the data rather than procedures or functions because it works as a **real world**.
POP follows **Top Down approach**.	OOP follows **Bottom Up approach**.
POP does not have any access specifier.	OOP has access specifiers named Public, Private, Protected, etc.
In POP, Data can move freely from function to function in the system.	In OOP, objects can move and communicate with each other through member functions.
To add new data and function in POP is not so easy.	OOP provides an easy way to add new data and function.

In POP, Most function uses Global data for sharing that can be accessed freely from function to function in the system.	In OOP, data cannot move easily from function to function, it can be kept public or private so we can control the access of data.
POP does not have any proper way for hiding data so it is **less secure**.	OOP provides Data Hiding so provides **more security**.
In POP, Overloading is not possible.	In OOP, overloading is possible in the form of Function Overloading and Operator Overloading.
Example of POP are: C, VB, FORTRAN, Pascal.	Example of OOP are : C++, JAVA, VB.NET, C#.NET.

.Net Framework Design Principle

Now in this .Net Architecture tutorial, we will learn the design principles of .Net framework. The following design principles of the .Net framework is what makes it very relevant to create .Net based applications.

1) Interoperability – The .Net framework provides a lot of backward support. Suppose if you had an application built on an older version of the .Net framework, say 2.0. And if you tried to run the same application on a machine which had the higher version of the .Net framework, say 3.5. The application would still work. This is because with every release, Microsoft ensures that older framework versions gel well with the latest version.

2) Portability – Applications built on the .Net framework can be made to work on any Windows platform. And now in recent times, Microsoft is also envisioning to make Microsoft products work on other platforms, such as iOS and Linux.

3) Security – The .NET Framework has a good security mechanism. The inbuilt security mechanism helps in both validation and verification of applications. Every application can explicitly define their security mechanism. Each security mechanism is used to grant the user access to the code or to the running program.

4) Memory management – The Common Language runtime does all the work or memory management. The .Net framework has all the capability to see those resources, which are not used by a running program. It would then release those resources accordingly. This is done via a program called the "Garbage Collector" which runs as part of the .Net framework. The

garbage collector runs at regular intervals and keeps on checking which system resources are not utilized, and frees them accordingly.

5) Simplified deployment – The .Net framework also have tools, which can be used to package applications built on the .Net framework. These packages can then be distributed to client machines. The packages would then automatically install the application.

Access Modifiers / Specifies

All types and type members have an accessibility level. The accessibility level controls whether they can be used from other code in your assembly or other assemblies. An assembly is a *.dll* or *.exe* created by compiling one or more *.cs* files in a single compilation. Use the following access modifiers to specify the accessibility of a type or member when you declare it:

- **public**: The type or member can be accessed by any other code in the same assembly or another assembly that references it. The accessibility level of public members of a type is controlled by the accessibility level of the type itself.
- **private**: The type or member can be accessed only by code in the same class or struct.
- **protected**: The type or member can be accessed only by code in the same class, or in a class that is derived from that class.
- **internal**: The type or member can be accessed by any code in the same assembly, but not from another assembly. In other words, internal types or members can be accessed from code that is part of the same compilation.
- **protected internal**: The type or member can be accessed by any code in the assembly in which it's declared, or from within a derived class in another assembly.
- **private protected**: The type or member can be accessed by types derived from the class that are declared within its containing assembly.

Exception Handling

An exception is a problem that arises during the execution of a program. A C# exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.

Exceptions provide a way to transfer control from one part of a program to another. C# exception handling is built upon four keywords: **try**, **catch**, **finally**, and **throw**.

- **try** − A try block identifies a block of code for which particular exceptions is activated. It is followed by one or more catch blocks.
- **catch** − A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The catch keyword indicates the catching of an exception.
- **finally** − The finally block is used to execute a given set of statements, whether an exception is thrown or not thrown. For example, if you open a file, it must be closed whether an exception is raised or not.

- **throw** − A program throws an exception when a problem shows up. This is done using a throw keyword.

Syntax

Assuming a block raises an exception, a method catches an exception using a combination of the try and catch keywords. A try/catch block is placed around the code that might generate an exception. Code within a try/catch block is referred to as protected code, and the syntax for using try/catch looks like the following −

```
try {
   // statements causing exception
} catch( ExceptionName e1 ) {
   // error handling code
} catch( ExceptionName e2 ) {
   // error handling code
} catch( ExceptionName eN ) {
   // error handling code
} finally {
   // statements to be executed
}
```

You can list down multiple catch statements to catch different type of exceptions in case your try block raises more than one exception in different situations.

Exception Classes in C#

C# exceptions are represented by classes. The exception classes in C# are mainly directly or indirectly derived from the **System.Exception** class. Some of the exception classes derived from the System.Exception class are the **System.ApplicationException** and **System.SystemException** classes.

The **System.ApplicationException** class supports exceptions generated by application programs. Hence the exceptions defined by the programmers should derive from this class.

The **System.SystemException** class is the base class for all predefined system exception.

The following table provides some of the predefined exception classes derived from the Sytem.SystemException class −

Sr.No.	Exception Class & Description
1	**System.IO.IOException** Handles I/O errors.
2	**System.IndexOutOfRangeException** Handles errors generated when a method refers to an array index out of range.
3	**System.ArrayTypeMismatchException** Handles errors generated when type is mismatched with the array type.

4	**System.NullReferenceException** Handles errors generated from referencing a null object.
5	**System.DivideByZeroException** Handles errors generated from dividing a dividend with zero.
6	**System.InvalidCastException** Handles errors generated during typecasting.
7	**System.OutOfMemoryException** Handles errors generated from insufficient free memory.
8	**System.StackOverflowException** Handles errors generated from stack overflow.

Handling Exceptions

C# provides a structured solution to the exception handling in the form of try and catch blocks. Using these blocks the core program statements are separated from the error-handling statements.

These error handling blocks are implemented using the **try**, **catch**, and **finally** keywords. Following is an example of throwing an exception when dividing by zero condition occurs −

```
using System;

namespace ErrorHandlingApplication {
  class DivNumbers {
    int result;

    DivNumbers() {
      result = 0;
    }
    public void division(int num1, int num2) {
      try {
        result = num1 / num2;
      } catch (DivideByZeroException e) {
        Console.WriteLine("Exception caught: {0}", e);
      } finally {
        Console.WriteLine("Result: {0}", result);
      }
    }
    static void Main(string[] args) {
      DivNumbers d = new DivNumbers();
      d.division(25, 0);
      Console.ReadKey();
```

```
    }
  }
}
```

When the above code is compiled and executed, it produces the following result −

```
Exception caught: System.DivideByZeroException: Attempted to divide by zero.
at ...
Result: 0
```

Thread

A **thread** is defined as the execution path of a program. Each thread defines a unique flow of control. If your application involves complicated and time consuming operations, then it is often helpful to set different execution paths or threads, with each thread performing a particular job.

Threads are **lightweight processes**. One common example of use of thread is implementation of concurrent programming by modern operating systems. Use of threads saves wastage of CPU cycle and increase efficiency of an application.

So far we wrote the programs where a single thread runs as a single process which is the running instance of the application. However, this way the application can perform one job at a time. To make it execute more than one task at a time, it could be divided into smaller threads.

Thread Life Cycle

The life cycle of a thread starts when an object of the System.Threading.Thread class is created and ends when the thread is terminated or completes execution.

Following are the various states in the life cycle of a thread −

- **The Unstarted State** − It is the situation when the instance of the thread is created but the Start method is not called.
- **The Ready State** − It is the situation when the thread is ready to run and waiting CPU cycle.
- **The Not Runnable State** − A thread is not executable, when
    - Sleep method has been called
    - Wait method has been called
    - Blocked by I/O operations
- **The Dead State** − It is the situation when the thread completes execution or is aborted.

The Main Thread

In C#, the **System.Threading.Thread** class is used for working with threads. It allows creating and accessing individual threads in a multithreaded application. The first thread to be executed in a process is called the **main** thread.

When a C# program starts execution, the main thread is automatically created. The threads created using the **Thread** class are called the child threads of the main thread. You can access a thread using the **CurrentThread** property of the Thread class.

The following program demonstrates main thread execution −

```
using System;
using System.Threading;

namespace MultithreadingApplication {
```

```
class MainThreadProgram {
   static void Main(string[] args) {
      Thread th = Thread.CurrentThread;
      th.Name = "MainThread";

      Console.WriteLine("This is {0}", th.Name);
      Console.ReadKey();
   }
}
}
```

When the above code is compiled and executed, it produces the following result −

This is MainThread

Properties and Methods of the Thread Class

The following table shows some most commonly used **properties** of the **Thread** class −

Sr.No.	Property & Description
1	**CurrentContext** Gets the current context in which the thread is executing.
2	**CurrentCulture** Gets or sets the culture for the current thread.
3	**CurrentPrinciple** Gets or sets the thread's current principal (for role-based security).
4	**CurrentThread** Gets the currently running thread.
5	**CurrentUICulture** Gets or sets the current culture used by the Resource Manager to look up culture-specific resources at run-time.
6	**ExecutionContext** Gets an ExecutionContext object that contains information about the various contexts of the current thread.
7	**IsAlive** Gets a value indicating the execution status of the current thread.

8	**IsBackground** Gets or sets a value indicating whether or not a thread is a background thread.
9	**IsThreadPoolThread** Gets a value indicating whether or not a thread belongs to the managed thread pool.
10	**ManagedThreadId** Gets a unique identifier for the current managed thread.
11	**Name** Gets or sets the name of the thread.
12	**Priority** Gets or sets a value indicating the scheduling priority of a thread.
13	**ThreadState** Gets a value containing the states of the current thread.

Windows Form Application

Introduction to Win Forms:

Windows Forms (WinForms) is a graphical (GUI) class library included as a part of Microsoft

.NET Framework, providing a platform to write rich client applications for desktop, laptop, and tablet PCs.

A Windows Forms application is an event-driven application supported by Microsoft's .NET Framework. Unlike a batch program, it spends most of its time simply waiting for the user to do something, such as fill in a text box or click a button.

All visual elements in the Windows Forms class library derive from the Control class. This provides a minimal functionality of a user interface element such as location, size, color, font, text, as well as common events like click and drag/drop.

Basic Controls:

The following table lists some of the commonly used controls:

S.N.	Widget & Description
1	**Forms** The container for all the controls that make up the user interface.
2	**TextBox** It represents a Windows text box control.
3	**Label** It represents a standard Windows label.

4	**Button** It represents a Windows button control.
5	**ListBox** It represents a Windows control to display a list of items.
6	**ComboBox** It represents a Windows combo box control.
7	**RadioButton**

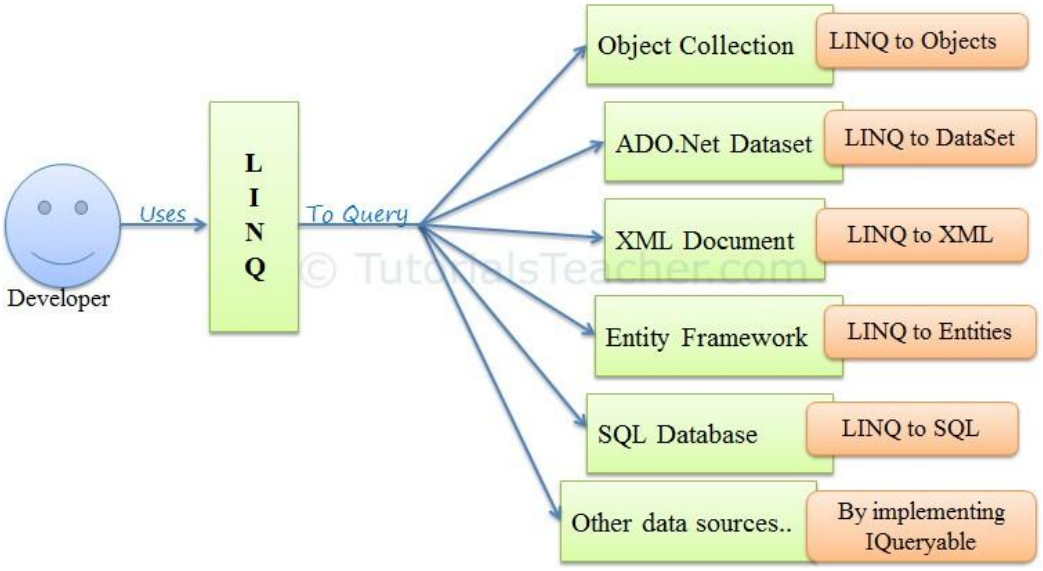
	It enables the user to select a single option from a group of choices when paired with other RadioButton controls.
8	**CheckBox** It represents a Windows CheckBox.
9	**PictureBox** It represents a Windows picture box control for displaying an image.
10	**ProgressBar** It represents a Windows progress bar control.
11	**ScrollBar** It Implements the basic functionality of a scroll bar control.
12	**DateTimePicker** It represents a Windows control that allows the user to select a date and a time and to display the date and time with a specified format.
13	**TreeView** It displays a hierarchical collection of labeled items, each represented by a TreeNode.

14	**ListView** It represents a Windows list view control, which displays a collection of items that can be displayed using one of four different views.

Introduction to LINQ

LINQ (Language Integrated Query) is uniform query syntax in C# to retrieve data from different sources and formats. It is integrated in C#, thereby eliminating the mismatch between programming languages and databases, as well as providing a single querying interface for different types of data sources.

For example, SQL is a Structured Query Language used to save and retrieve data from a database. In the same way, LINQ is a structured query syntax built in C# to retrieve data from different types of data sources such as collections, ADO.Net DataSet, XML Docs, web service and MS SQL Server and other databases.



LINQ queries return results as objects. It enables you to uses object-oriented approach on the result set and not to worry about transforming different formats of results into objects.

Working with Databases

Comparison between ADO and ADO.NET:

ADO is a Microsoft technology. It stands for ActiveX Data Objects. It is a Microsoft Active- X component. ADO is automatically installed with Microsoft IIS. It is a programming interface to access data in a database

ADO.NET is a set of classes that expose data access services for .NET Framework programmers. ADO.NET provides a rich set of components for creating distributed, data- sharing applications. It is an integral part of the .NET Framework, providing access to relational, XML, and application data. ADO.NET supports a variety of development needs, including the creation of front-end database clients and middle-tier business objects used by applications, tools, languages, or Internet browsers.

ADO : ActiveX Data Objects **and ADO.Net are two different ways to access database in Microsoft.**

ADO	**ADO.Net**
ADO is base on **COM** : **Component Object Modelling** based.	ADO.Net is based on **CLR** : **Common Language Runtime** based.
ADO stores data in **binary** format.	ADO.Net stores data in **XML** format i.e. parsing of data.
ADO can't be integrated with XML because ADO have limited access of XML.	ADO.Net can be integrated with XML as having robust support of XML.
In ADO, data is provided by **RecordSet**.	In ADO.Net data is provided by **DataSet** or **DataAdapter**.
ADO is **connection** oriented means it requires continuous active connection.	ADO.Net is **disconnected**, does not need continuous connection.

ADO gives rows as single table view, it scans sequentially the rows using **MoveNext** method.	ADO.Net gives rows as collections so you can access any record and also can go through a table via loop.
In ADO, You can create only **Client** side cursor.	In ADO.Net, You can create both **Client & Server** side cursor.
Using a single connection instance, ADO can not handle multiple transactions.	Using a single connection instance, ADO.Net can handle multiple transactions.

Working with Connection, Command:

Working with Connection:

Process of creating connection:

For SQL Server

Note: Sql Server must be installed

```
String conn_str; SqlConnection
connection;

conn_str = "Data Source=DESKTOP-EG4ORHN\SQLEXPRESS; Initial Catalog=billing;
            User ID=sa;Password=24518300";

connection = New SqlConnection(conn_str);
```

(Here, DESKTOP-EG4ORHN\SQLEXPRESS refers to a data source and billing refers to database name)

Working with Command:

```
SqlConnection connection;
SqlCommand command;

String conn_str="Data Source=Raazu\\SQLEXPRESS; Initial
            Catalog=billing; User ID=sa;Password=24518300";

connection = new SqlConnection(conn_str); connection.Open();
```

```
String sql = "insert into tblCustomer(name,address) values(“Raaju”,
                    “Birtamode”)";

        command = new SqlCommand(sql, connection);
        command.ExecuteNonQuery(); connection.Close();
```

DataReader, DataAdaper, Dataset and Datatable :

DataReader is used to read the data from database and it is a read and forward only connection oriented architecture during fetch the data from database. DataReader will fetch the data very fast when compared with dataset. Generally we will use ExecuteReader object to bind data to dataReader.

//Example

SqlDataReader sdr = cmd.ExecuteReader();

DataReader

- Holds the connection open until you are finished (don't forget to close it!).
- Can typically only be iterated over once
- Is not as useful for updating back to the database

DataSet is a disconnected orient architecture that means there is no need of active connections during work with datasets and it is a collection of DataTables and relations between tables. It is used to hold multiple tables with data. You can select data form tables, create views based on table and ask child rows over relations. Also DataSet provides you with rich features like saving data as XML and loading XML data.

//Example

```
DataSet ds = new DataSet();
  da.Fill(ds,"user");
```

DataAdapter will acts as a Bridge between DataSet and database. This data adapter object is used to read the data from database and bind that data to dataset. Data adapter is a disconnected oriented architecture.

//Example

```
SqlDataAdapter sda = new SqlDataAdapter(cmd);
  DataSet ds = new DataSet();

  sda.Fill(ds,"user");
```

DataAdapter

- Lets you close the connection as soon it's done loading data, and may even close it for you

automatically

- All of the results are available in memory
- You can iterate over it as many times as you need, or even look up a specific record by index
- Has some built-in faculties for updating back to the database.

DataTable represents a single table in the database. It has rows and columns. There is no much difference between dataset and datatable, dataset is simply the collection of datatables.

```
//Example

SqlDataAdapter sda = new SqlDataAdapter(cmd);

DataTable dt = new DataTable();
    sda.Fill(dt);
```

Difference between DataReader and DataAdapter:

1) A DataReader is an object returned from the ExecuteReader method of a DbCommand object. It is a forward-only cursor over the rows in each result set. Using a DataReader, you can access each column of the result set, read all rows of the set, and advance to the next result set if there are more than one.

A DataAdapter is an object that contains four DbCommand objects: one each for SELECT, INSERT, DELETE and UPDATE commands. It mediates between these commands and a DataSet though the Fill and Update methods.

2) DataReader is a faster way to retrieve the records from the DB. DataReader reads the column. DataReader demands live connection but DataAdapter needs disconnected approach.

3) Data reader is an object through which you can read a sequential stream of data. it's a forward only data wherein you cannot go back to read previous data. data set and data adapter object help us to work in disconnected mode. data set is an in cache memory representation of tables. the data is filled from the data source to the data set thro' the data adapter. once the table in the dataset is modified, the changes are broadcast to the database back throw; the data adapter.

DataReader vs DataAdapter

	DataReader	DataAdapter
1	Can Access one table at a Time.	Can Access Multiple Table at a Time.

2	Handles DataBase Tables.	Handles XML Files, Text Files, DataBase Tables.
3	Only READ Operation.	Both Read/Write Operations.
4	DataBase Connection Mode.	DataBase DisConnection Mode.
5	I. No Storage Capacity. II. Faster than DataAdapter	I. Temporary Storage Capacity. II. Less Faster than DataReader

Connected vs Disconnected

Connected	Disconnected
It is connection oriented.	It is dis_connection oriented.
Datareader	DataSet
Connected methods gives faster performance	Disconnected get low in speed and performance.
connected can hold the data of single table	disconnected can hold multiple tables of data
connected you need to use a read only forward only data reader	disconnected you cannot
Data Reader can't persist the data	Data Set can persist the data
It is Read only; we can't update the data.	We can update data

Web Applications using ASP.NET

A Visual Studio Web application is built around ASP.NET. ASP.NET is a platform — including design-time objects and controls and a run-time execution context — for developing and running applications on a Web server.

ASP.NET Web applications run on a Web server configured with Microsoft Internet

Information Services (IIS). However, you do not need to work directly with IIS. You can program IIS facilities using ASP.NET classes, and Visual Studio handles file management tasks such as creating IIS applications when needed and providing ways for you to deploy your Web applications to IIS.

ASP.NET Architecture

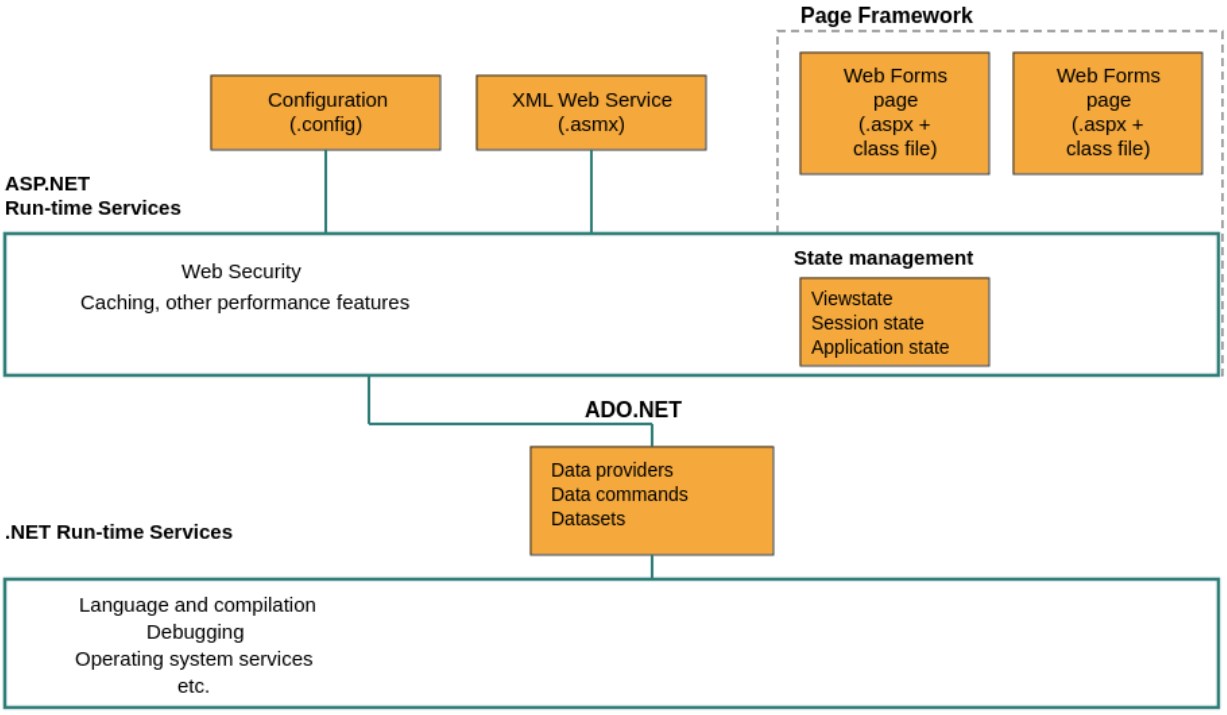


Fig. ASP.NET Architecture

ASP.NET Run-time Services (middle layer)
This layer supports the page framework and includes:

- **Configuration (.config)** — app settings and configuration files
- **XML Web Service (.asmx)** — exposes web service endpoints
- **State management** — handles Viewstate, Session state, and Application state so data persists across requests
- **Web Security, Caching, and other performance features** — cross-cutting services like authentication, authorization, and output caching

ADO.NET
Sits between the ASP.NET layer and the .NET runtime, providing **data providers, data commands, and datasets** — the components used to connect to and interact with databases.

.NET Run-time Services (bottom layer)
The foundational layer everything else runs on top of, covering:

- Language and compilation
- Debugging
- Operating system services, etc.

How it flows: Configuration and XML Web Services feed into the ASP.NET runtime layer, which handles security/caching/state for the Page Framework above it. That layer connects down through ADO.NET (for data access) into the core .NET runtime services that handle compilation, debugging, and OS-level operations.

Elements of ASP.NET Web Applications

Creating ASP.NET Web applications involve working with many of the same elements you use in any desktop or client-server application. These include:

- **Project management features** When creating an ASP.NET Web application, you need to keep track of the files you need, which ones need to be compiled, and which need to be deployed.
- **User interface** Your application typically presents information to users; in an ASP.NET Web application, the user interface is presented in Web Forms pages, which send output to a browser. Optionally, you can create output tailored for mobile devices or other Web appliances.
- **Components** Many applications include reusable elements containing code to perform specific tasks. In Web applications, you can create these components as XML Web services, which makes them callable across the Web from a Web application, another XML Web service, or a Windows Form, for example.
- **Data** Most applications require some form of data access. In ASP.NET Web applications, you can use ADO.NET, the data services that are part of the .NET Framework.
- **Security, performance, and other infrastructure features** As in any application, you must implement security to prevent unauthorized use, test and debug the application, tune its performance, and perform other tasks not directly related to the application's primary function.

Different Types of form controls in ASP.NET

Button Controls

ASP.NET provides three types of button control:

- **Button** : It displays text within a rectangular area.

- **Link Button** : It displays text that looks like a hyperlink.
- **Image Button** : It displays an image.

```
<asp:Button ID="Button1" runat="server" onclick="Button1_Click"
Text="Click" / >
```

Text Boxes and Labels

Text box controls are typically used to accept input from the user. A text box control can accept one or more lines of text depending upon the settings of the Text Mode attribute.

Label controls provide an easy way to display text which can be changed from one execution of a page to the next. If you want to display text that does not change, you use the literal text.

```
<asp:TextBox ID="txtstate" runat="server" ></asp:TextBox>
```

Check Boxes and Radio Buttons

A check box displays a single option that the user can either check or uncheck and radio buttons present a group of options from which the user can select just one option.

To create a group of radio buttons, you specify the same name for the **GroupName** attribute of each radio button in the group. If more than one group is required in a single form, then specify a different group name for each group.

If you want check box or radio button to be selected when the form is initially displayed, set its Checked attribute to true. If the Checked attribute is set to true for multiple radio buttons in a group, then only the last one is considered as true.

```
<asp:CheckBox ID= "chkoption" runat= "Server">
</asp:CheckBox>
```

```
<asp:RadioButton ID= "rdboption" runat= "Server">
</asp: RadioButton>
```

List box Control

These control let a user choose from one or more items from the list. List boxes and drop- down lists contain one or more list items. These lists can be loaded either by code or by the **ListItemCollection** editor.

```
<asp:ListBox ID="ListBox1" runat="server">
</asp:ListBox>
```

HyperLink Control

The HyperLink control is like the HTML <a> element.

```
<asp:HyperLink ID="HyperLink1" runat="server">
   HyperLink
</asp:HyperLink>
```

Image Control

The image control is used for displaying images on the web page, or some alternative text, if the image is not available.

```
<asp:Image ID="Image1" ImageUrl="url" runat="server">
```

Drop down List Control

```
<asp:DropDownList ID="DropDownList1" runat="server"
</asp:DropDownList>
```

Launch another form on button click

```
protected void btnSelect_Click(object sender, EventArgs e)
        {
            Response.Redirect("AnotherForm.aspx");
        }
```

Example 1 – Creating a basic form in ASP.Net

Name:

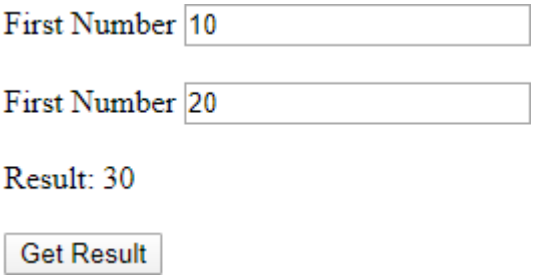
Address:

Submit

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="MyForm.aspx.cs"
Inherits="WebApplication1.MyForm" %>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
   <title>This is my first application</title>
```

```
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <asp:Label ID="lblName" runat="server" Text="Name:"></asp:Label>
      <asp:TextBox ID="txtName" runat="server"></asp:TextBox>
      <br />
      <asp:Label ID="lblAddress" runat="server" Text="Address:"></asp:Label>
      <asp:TextBox ID="txtAddress" runat="server"></asp:TextBox>
      <br />
      <asp:Button ID="btnSubmit" runat="server" Text="Submit" />
    </div>
  </form>
</body>
</html>
```

Example – 2 (Handling Events)



EventHandling.aspx

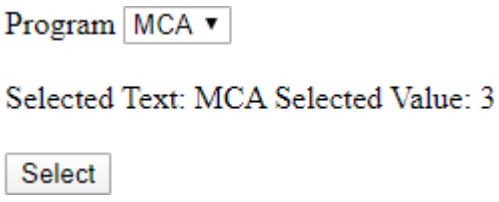
```
<form id="form1" runat="server">
    <div>
        <asp:Label ID="Label1" runat="server" Text="First Number">
            </asp:Label>
        <asp:TextBox ID="txtFirst" runat="server"></asp:TextBox>
                    <br/><br/>
        <asp:Label ID="Label2" runat="server" Text="First Number">
                </asp:Label>
```

```
<asp:TextBox ID="txtSecond" runat="server"></asp:TextBox>
                <br/><br/>
<asp:Label ID="lblResult" runat="server" Text="Result:">
                </asp:Label> <br/><br/>
<asp:Button ID="btnSubmit" runat="server" Text="Get Result"
                onClick="btnSubmit_Click"/>
</div>
</form>
```

EventHandling.cs

```
protected void btnSubmit_Click(object sender, EventArgs e)
        {
            int first = Convert.ToInt32(txtFirst.Text); int
            second = Convert.ToInt32(txtSecond.Text); int res
            = first + second;

            lblResult.Text = "Result: " + res;
        }
```

Example 3 – Using Drop down list



Dropdown.aspx

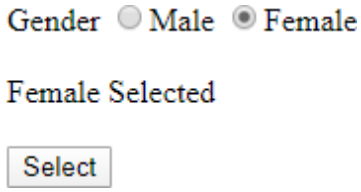
```
<form id="form1" runat="server">
    <div>
        <asp:Label ID="Label1" runat="server" Text="Program">
            </asp:Label>
        <asp:DropDownList ID="dropProgram" runat="server">
                </asp:DropDownList>
        <br/><br/>
        <asp:Label ID="lblSelected" runat="server" Text="Selected:">
            </asp:Label>
        <br/><br/>
        <asp:Button ID="btnSelect" runat="server" Text="Select"
            OnClick="btnSelect_Click" />
    </div>
    </form>
```

Dropdown.cs

```
private void LoadData()
        {
            List<ListItem> mylist=new List<ListItem>();
            mylist.Add(new ListItem("BCA","1"));
            mylist.Add(new ListItem("BBA","2"));
            mylist.Add(new ListItem("MCA","3"));
            mylist.Add(new ListItem("MBA","4"));
            dropProgram.Items.AddRange(mylist.ToArray());
        }

protected void btnSelect_Click(object sender, EventArgs e)
```

```
{
    string text = dropProgram.SelectedItem.ToString(); string
    value = dropProgram.SelectedValue; lblSelected.Text =
    "Selected Text: " + text + " Selected

        Value: " + value;

}
```

Example – 4 Using Radio button



example.aspx

```
<form id="form1" runat="server">

    <div>

        <asp:Label ID="Label1" runat="server" Text="Gender"></asp:Label>

        <asp:RadioButton ID="radioMale" Text="Male" GroupName="gender"
            runat="server" />

        <asp:RadioButton ID="radioFemale" Text="Female"
            GroupName="gender" runat="server" />

        <br/><br/>

        <asp:Label ID="lblSelected" runat="server" Text="Selected
```

```
            Radio:"> </asp:Label>

        <br/><br/>

        <asp:Button ID="btnSelect" runat="server" Text="Select"
                    OnClick="btnSelect_Click" />

    </div>

    </form>
```

example.cs

```
protected void btnSelect_Click(object sender, EventArgs e)

        {

            if (radioMale.Checked)  lblSelected.Text
                = "Male Selected";

            else

                lblSelected.Text = "Female Selected";

        }
```

Example – 5 Using Check box

Gender ☐ Male ☑ Female

Female Selected

Select

example.aspx

```
<form id="form1" runat="server">

    <div>

        <asp:Label ID="Label1" runat="server" Text="Gender"></asp:Label>

        <asp:CheckBox ID="chkMale" Text="Male" GroupName="gender"
                    runat="server" />

        <asp:CheckBox ID="chkFemale" Text="Female" GroupName="gender"
                    runat="server" />
```

```
        <br/><br/>

        <asp:Label ID="lblSelected" runat="server" Text="Selected
            Radio:"> </asp:Label>

        <br/><br/>

        <asp:Button ID="btnSelect" runat="server" Text="Select"
                    OnClick="btnSelect_Click" />

    </div>

    </form>
```

example.cs

```
protected void btnSelect_Click(object sender, EventArgs e)

        {

            if (chkMale.Checked)

                lblSelected.Text = "Male Selected"; else

                lblSelected.Text = "Female Selected";

        }
```

Validation Controls in ASP.NET

An important aspect of creating ASP.NET Web pages for user input is to be able to check that the information users enter is valid. ASP.NET provides a set of validation controls that provide an easy-to-use but powerful way to check for errors and, if necessary, display messages to the user.

There are six types of validation controls in ASP.NET

- RequiredFieldValidation Control
- CompareValidator Control
- RangeValidator Control
- RegularExpressionValidator Control
- CustomValidator Control
- ValidationSummary

The below table describes the controls and their work:

Validation Control	**Description**
RequiredFieldValidation	Makes an input control a required field
CompareValidator	Compares the value of one input control to the value of another input control or to a fixed value
RangeValidator	Checks that the user enters a value that falls between two values
RegularExpressionValidator	Ensures that the value of an input control matches a specified pattern
CustomValidator	Allows you to write a method to handle the validation of the value entered
ValidationSummary	Displays a report of all validation errors occurred in a Web page

Example of Validation Controls

Name: Name is Required !

Email: sadf Email is invalid !

Class: 30 Class must be between (1-12)

Age: 101 Age must be less than 100 !

Submit

Errors:

- Name is Required !
- Email is invalid !
- Class must be between (1-12)
- Age must be less than 100 !

```
<form id="form1" runat="server">
  <div>
    <!-- Name Field -->
    <asp:Label ID="Label1" runat="server" Text="Name:"></asp:Label>
    <asp:TextBox ID="txtName" runat="server"></asp:TextBox>
    <asp:RequiredFieldValidator
      ID="validator1"
      runat="server"
      ControlToValidate="txtName"
      ErrorMessage="Name is Required !"
      ForeColor="Red">
    </asp:RequiredFieldValidator>
    <br /><br />

    <!-- Email Field -->
    <asp:Label ID="Label2" runat="server" Text="Email:"></asp:Label>
    <asp:TextBox ID="txtEmail" runat="server"></asp:TextBox>
    <asp:RegularExpressionValidator
      ID="validator2"
      runat="server"
      ControlToValidate="txtEmail"
      ErrorMessage="Email is invalid !"
      ValidationExpression="\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*"
      ForeColor="Red">
    </asp:RegularExpressionValidator>
    <br /><br />

    <!-- Class Field -->
```

```
        <asp:Label ID="Label3" runat="server" Text="Class:"></asp:Label>
        <asp:TextBox ID="txtClass" runat="server"></asp:TextBox>
        <asp:RangeValidator
            ID="validator3"
            runat="server"
            ControlToValidate="txtClass"
            MinimumValue="1"
            MaximumValue="12"
            Type="Integer"
            ErrorMessage="Class must be between (1-12)"
            ForeColor="Red">
        </asp:RangeValidator>
        <br /><br />

        <!-- Age Field -->
        <asp:Label ID="Label4" runat="server" Text="Age:"></asp:Label>
        <asp:TextBox ID="txtAge" runat="server"></asp:TextBox>
        <asp:CompareValidator
            ID="validator4"
            runat="server"
            ControlToValidate="txtAge"
            ValueToCompare="100"
            Operator="LessThan"
            Type="Integer"
            ErrorMessage="Age must be less than 100 !"
            ForeColor="Red">
        </asp:CompareValidator>
        <br /><br />

        <!-- Submit Action -->
        <asp:Button ID="btnSubmit" runat="server" Text="Submit" />
    </div>
    <br />

    <!-- Summary Section -->
    <asp:ValidationSummary
        ID="validator5"
        runat="server"
        HeaderText="Errors:"
        DisplayMode="BulletList"
        ShowSummary="true"
        ForeColor="Red" />
</form>
```

Difference Between DataReader, DataSet, DataAdapter and DataTable in C#

DataReader

DataReader is used to read the data from the database and it is a read and forward only connection-oriented architecture during fetch the data from database. DataReader will fetch the data very fast when compared with dataset. Generally, we will use ExecuteReader object to bind data to datareader.

To bind DataReader data to GridView we need to write the code like as shown below:

```
protected void BindGridview() {
   using(SqlConnection conn = new SqlConnection("Data Source=abc;Integrated
Security=true;Initial Catalog=Test")) {
      con.Open();
      SqlCommand cmd = new SqlCommand("Select UserName, First
Name,LastName,Location FROM Users", conn);
      SqlDataReader sdr = cmd.ExecuteReader();
      gvUserInfo.DataSource = sdr;
      gvUserInfo.DataBind();
      conn.Close();
   }
}
```

- Holds the connection open until you are finished (don't forget to close it!).
- Can typically only be iterated over once
- Is not as useful for updating back to the database

DataSet

DataSet is a disconnected orient architecture that means there is no need of active connections during work with datasets and it is a collection of DataTables and relations between tables. It is used to hold multiple tables with data. You can select data form tables, create views based on table and ask child rows over relations. Also, DataSet provides you with rich features like saving data as XML and loading XML data.

```
protected void BindGridview() {
   SqlConnection conn = new SqlConnection("Data Source=abc;Integrated
Security=true;Initial Catalog=Test");
   conn.Open();
   SqlCommand cmd = new SqlCommand("Select UserName, First
Name,LastName,Location FROM Users", conn);
   SqlDataAdapter sda = new SqlDataAdapter(cmd);
   DataSet ds = new DataSet();
   sda.Fill(ds);
   gvUserInfo.DataSource = ds;
   gvUserInfo.DataBind();
}
```

DataAdapter

DataAdapter will acts as a Bridge between DataSet and database. This dataadapter object is used to read the data from database and bind that data to dataset. Dataadapter is a disconnected oriented architecture. Check below sample code to see how to use DataAdapter in code:

```
protected void BindGridview() {
   SqlConnection con = new SqlConnection("Data Source=abc;Integrated
Security=true;Initial Catalog=Test");
   conn.Open();
   SqlCommand cmd = new SqlCommand("Select UserName, First
Name,LastName,Location FROM Users", conn);
   SqlDataAdapter sda = new SqlDataAdapter(cmd);
   DataSet ds = new DataSet();
   sda.Fill(ds);
   gvUserInfo.DataSource = ds;
   gvUserInfo.DataBind();
}
```

- Let's you close the connection as soon it's done loading data, and may even close it for you automatically
- All of the results are available in memory
- You can iterate over it as many times as you need, or even look up a specific record by index
- Has some built-in faculties for updating back to the database.

DataTable

DataTable represents a single table in the database. It has rows and columns. There is no much difference between dataset and datatable, dataset is simply the collection of datatables.

```
protected void BindGridview()
{
   SqlConnection con = new SqlConnection("Data Source=abc;Integrated
Security=true;Initial Catalog=Test");
   conn.Open();
   SqlCommand cmd = new SqlCommand("Select UserName, First
Name,LastName,Location FROM Users", conn);
   SqlDataAdapter sda = new SqlDataAdapter(cmd);
   DataTable dt = new DataTable();
   sda.Fill(dt);
   gridview1.DataSource = dt;
   gvidview1.DataBind();
}
```

ASP.NET MVC

ASP.NET MVC is an open-source software from Microsoft. Its web development framework combines the features of MVC (Model-View-Controller) architecture, the most up-to-date ideas and techniques from Agile development and the best parts of the existing ASP.NET platform. This tutorial provides a complete picture of the MVC framework and teaches you how to build an application using this tool.

Features of MVC

- It provides a clear separation of **business logic, UI logic, and input logic.**
- It offers full control over your HTML and URLs which makes it easy to design web application architecture.
- It is a powerful URL-mapping component using which we can build applications that have comprehensible and searchable URLs.
- It supports **Test Driven Development (TDD).**

Benefits of ASP.NET MVC

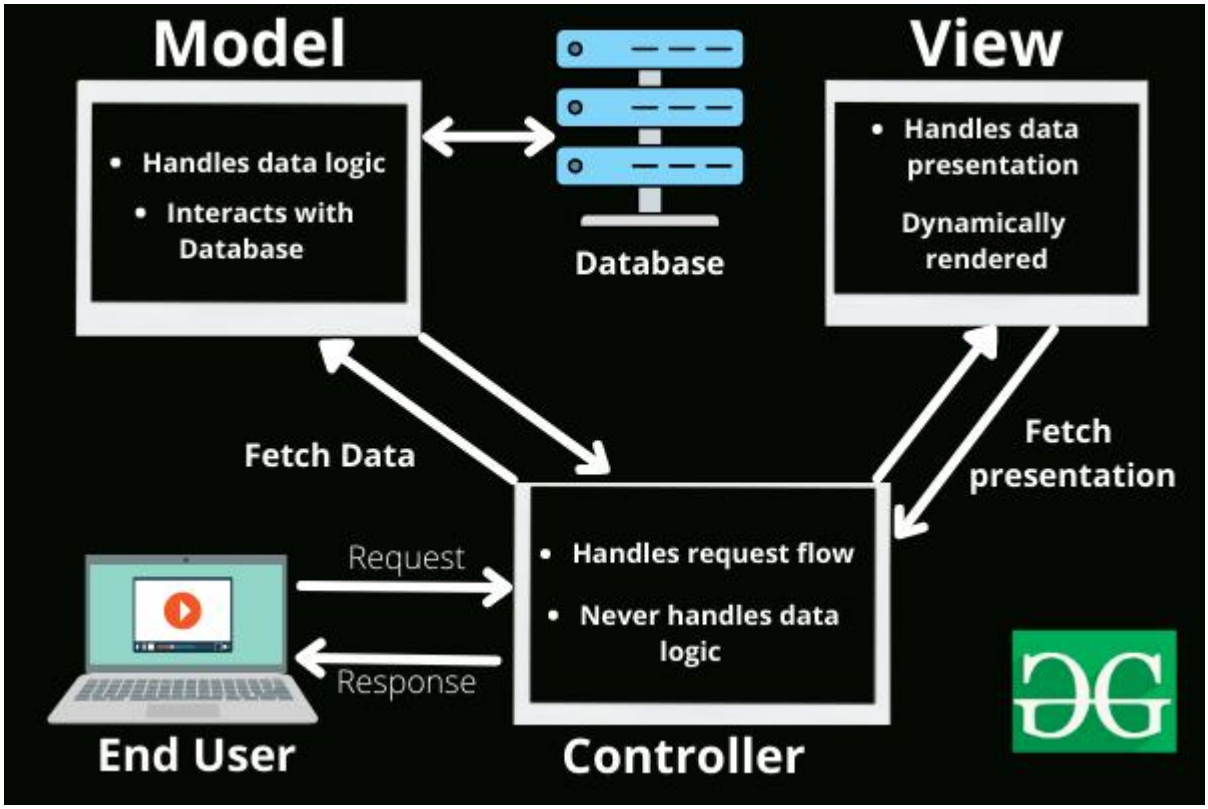
Following are the benefits of using ASP.NET MVC −

- Makes it easier to manage complexity by dividing an application into the model, the view, and the controller.
- Enables full control over the rendered HTML and provides a clean separation of concerns.
- Direct control over HTML also means better accessibility for implementing compliance with evolving Web standards.
- Facilitates adding more interactivity and responsiveness to existing apps.
- Provides better support for test-driven development (TDD).
- Works well for Web applications that are supported by large teams of developers and for Web designers who need a high degree of control over the application behavior.

Components of MVC

The MVC framework includes the following 3 components:

- Controller
- Model
- View



Controller:

The controller is the component that enables the interconnection between the views and the model so it acts as an intermediary. The controller doesn't have to worry about handling data logic; it just tells the model what to do. It processes all the business logic and incoming requests, manipulates data using the **Model** component, and interact with the **View** to render the final output.

Responsibilities:

- Receiving user input and interpreting it.
- Updating the Model based on user actions.
- Selecting and displaying the appropriate View.

Example: In a bookstore application, the Controller would handle actions such as searching for a book, adding a book to the cart, or checking out.

View:

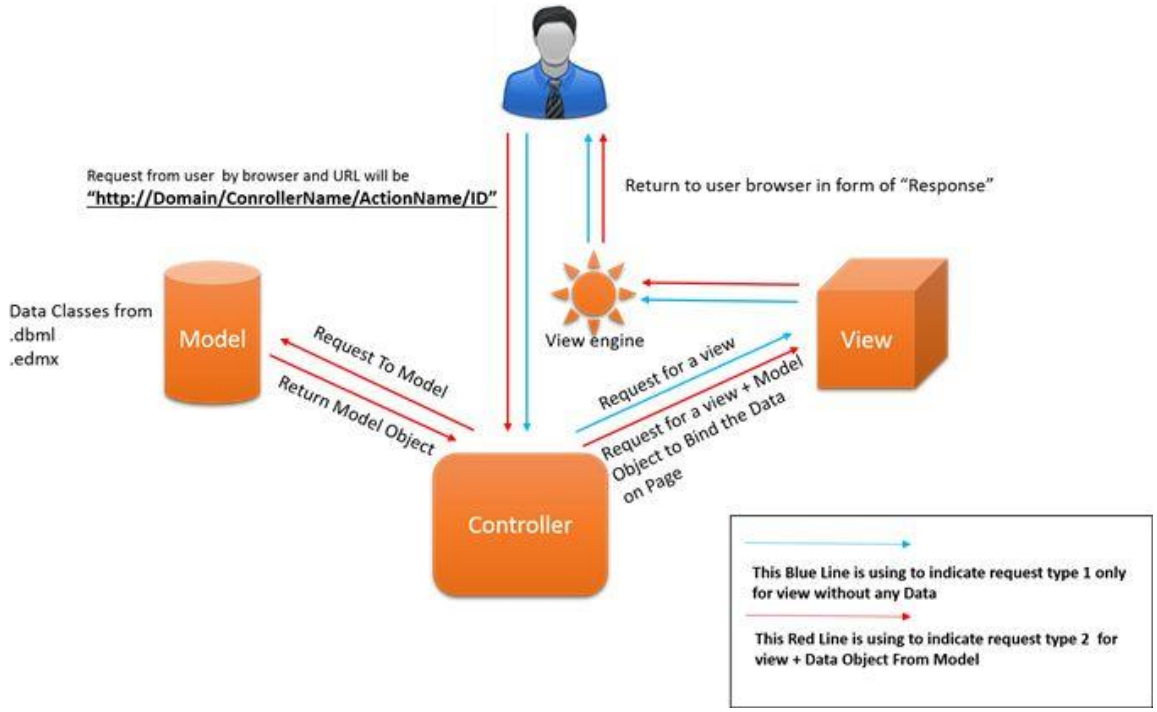
The **View** component is used for all the UI logic of the application. It generates a user

interface for the user. Views are created by the data which is collected by the model component but these data aren't taken directly but through the controller. It only interacts with the controller.

Responsibilities:

- Rendering data to the user in a specific format.
- Displaying the user interface elements.
- Updating the display when the Model changes.

Example: In a bookstore application, the View would display the list of books, book details, and provide input fields for searching or filtering books.



Model:

The **Model** component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data. It can add or retrieve data from the database. It responds to the controller's request because the controller can't interact with the database by itself. The model interacts with the database and gives the required data back to the controller.

Responsibilities:

- Managing data: CRUD (Create, Read, Update, Delete) operations.
- Enforcing business rules.
- Notifying the View and Controller of state changes.

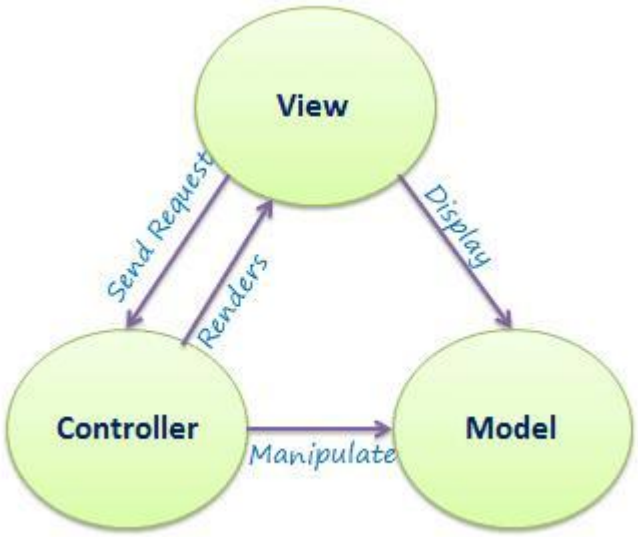
Example: In a bookstore application, the Model would handle data related to books, such as the book title, author, price, and stock level.

Advantages of MVC

- Codes are easy to maintain and they can be extended easily.
- The MVC **model** component can be tested separately.
- The components of MVC can be developed simultaneously.
- It reduces complexity by dividing an application into three units. **Model, view, and controller.**
- It supports **Test Driven Development (TDD).**
- It works well for Web apps that are supported by large teams of web designers and developers.
- This architecture helps to test components independently as all classes and objects are independent of each other
- **Search Engine Optimization (SEO)** Friendly.

Disadvantages of MVC

- It is difficult to read, change, test, and reuse this model
- It is not suitable for building small applications.
- The inefficiency of data access in view.
- The framework navigation can be complex as it introduces new layers of abstraction which requires users to adapt to the decomposition criteria of MVC.
- Increased complexity and Inefficiency of data

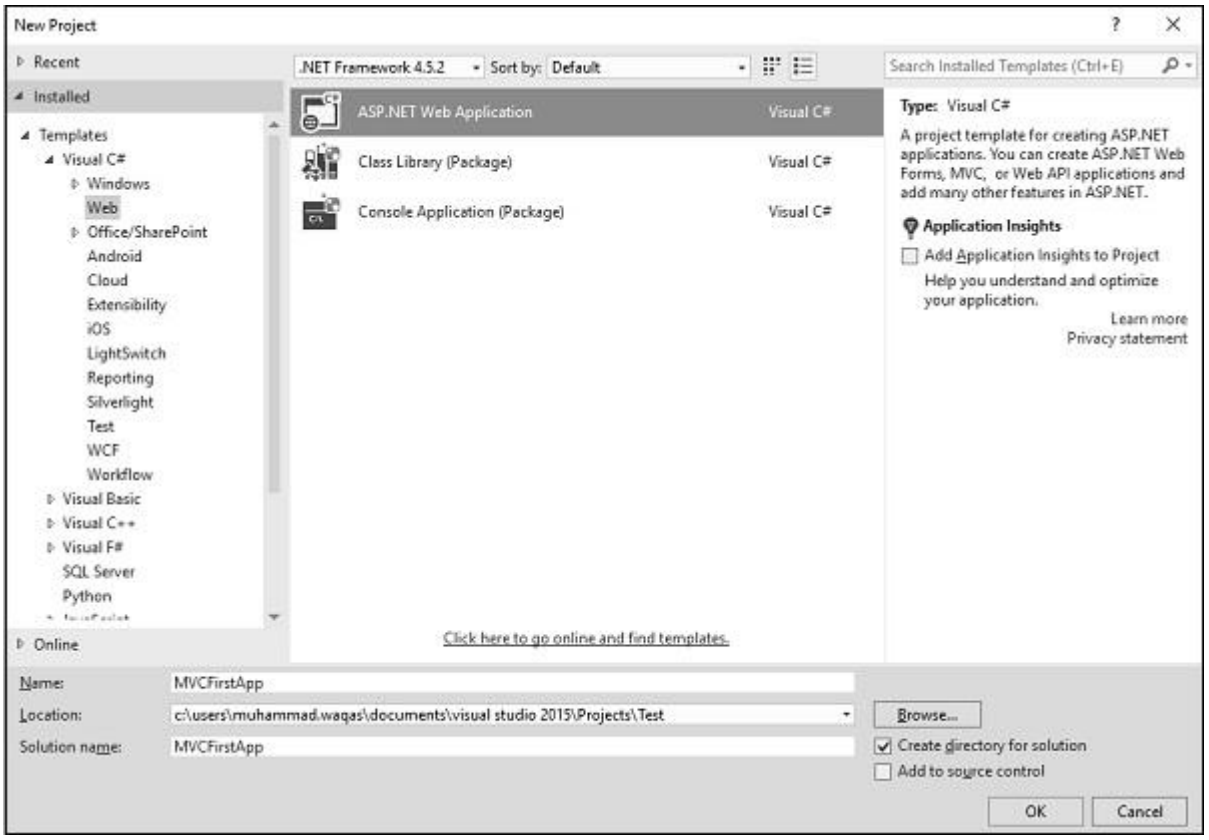


Create ASP.Net MVC Application

Following are the steps to create a project using project templates available in Visual Studio.

Step 1 − Open the Visual Studio. Click File → New → Project menu option.

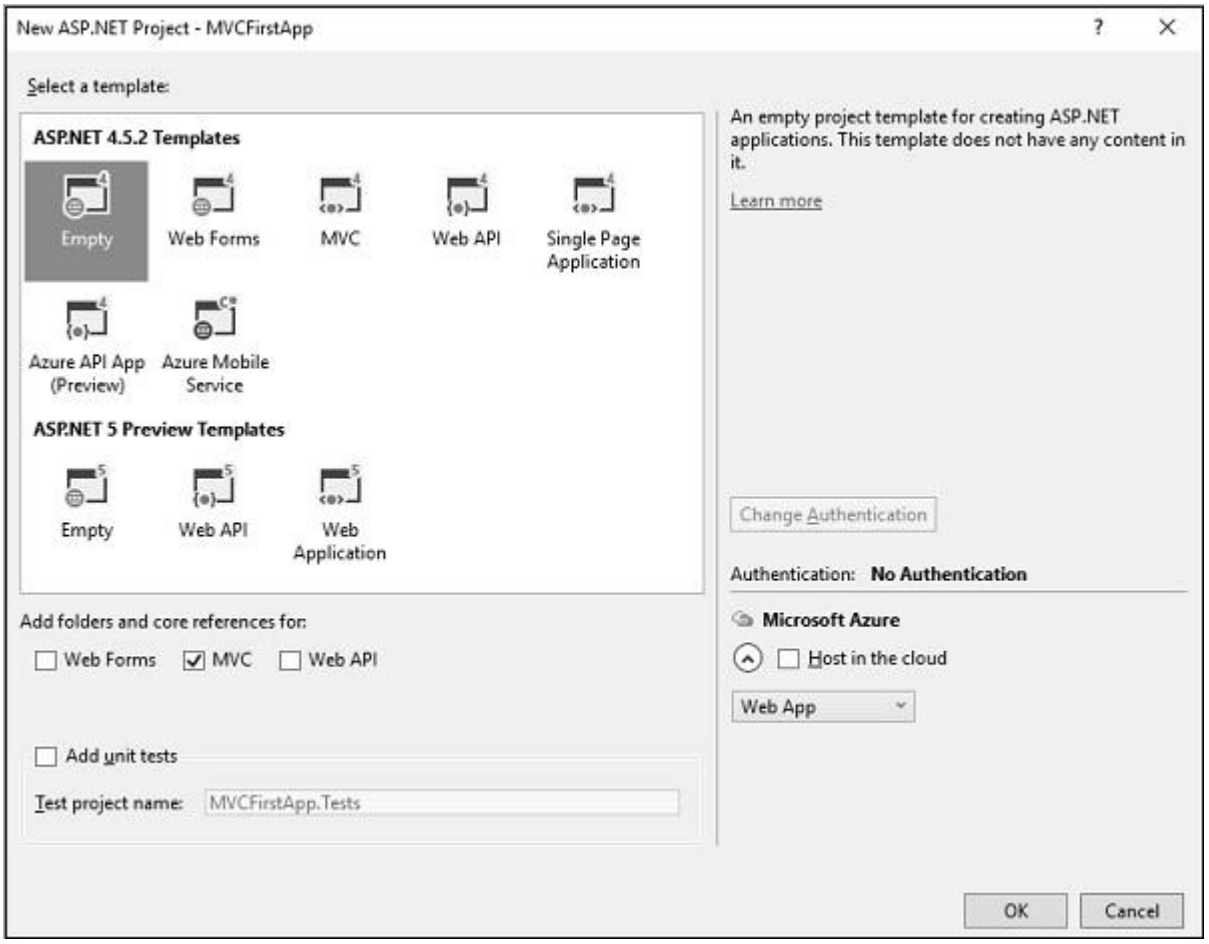
A new Project dialog opens.



Step 2 − From the left pane, select Templates → Visual C# → Web.

Step 3 − In the middle pane, select ASP.NET Web Application.

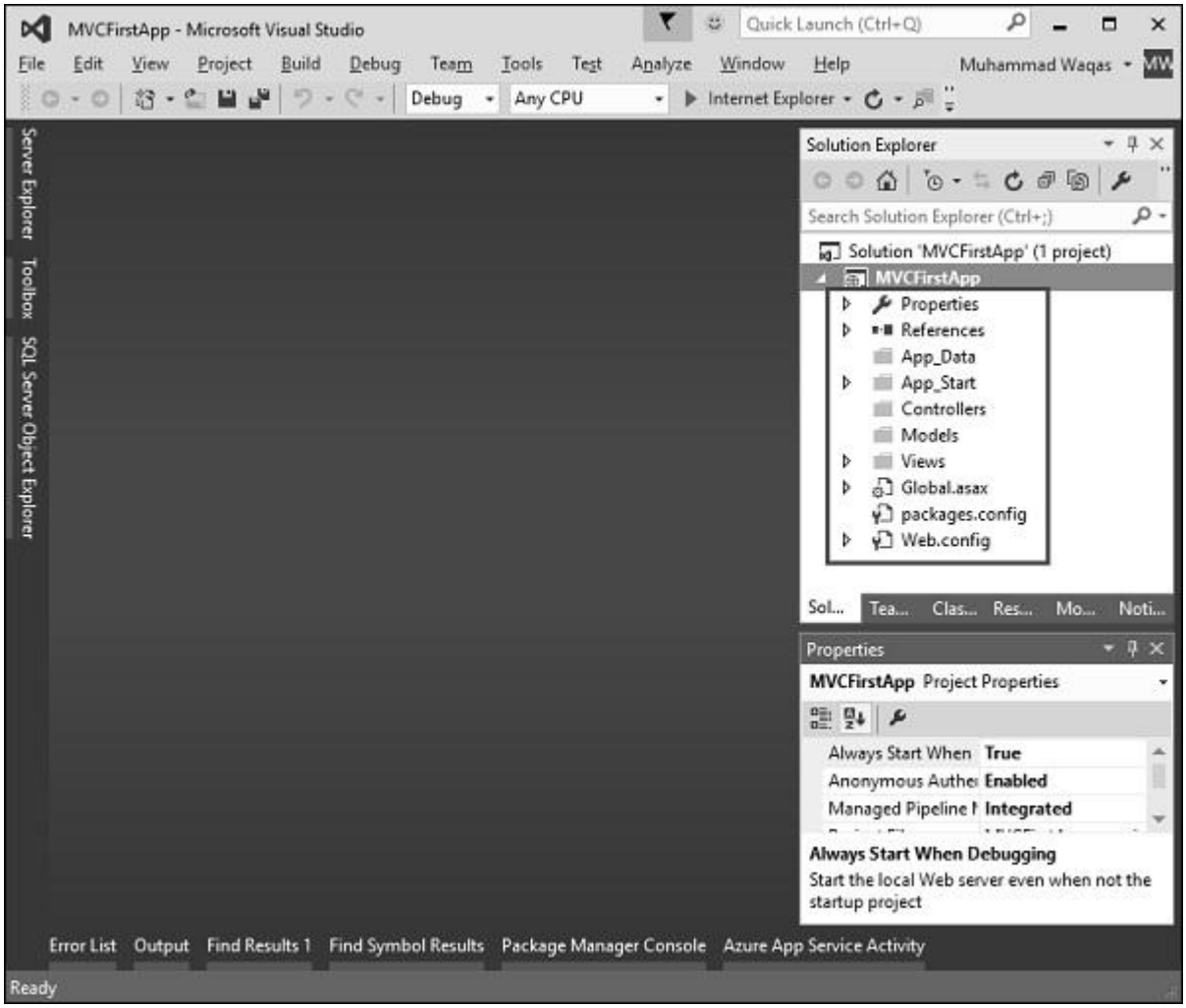
Step 4 − Enter the project name, MVCFirstApp, in the Name field and click ok to continue. You will see the following dialog which asks you to set the initial content for the ASP.NET project.



Step 5 − To keep things simple, select the Empty option and check the MVC checkbox in the Add folders and core references section. Click Ok.

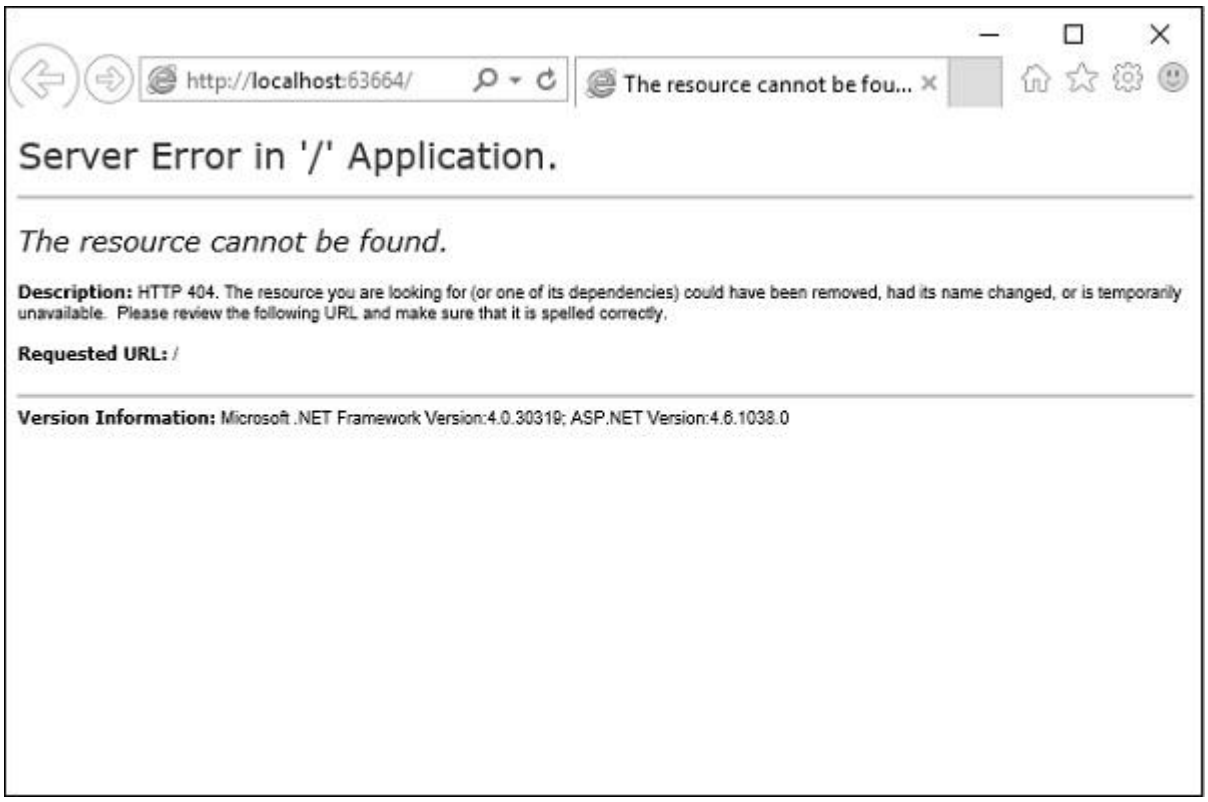
It will create a basic MVC project with minimal predefined content.

Once the project is created by Visual Studio, you will see a number of files and folders displayed in the Solution Explorer window.



As you know that we have created ASP.Net MVC project from an empty project template, so for the moment the application does not contain anything to run.

Step 6 − Run this application from Debug → Start Debugging menu option and you will see a **404 Not Found** Error.

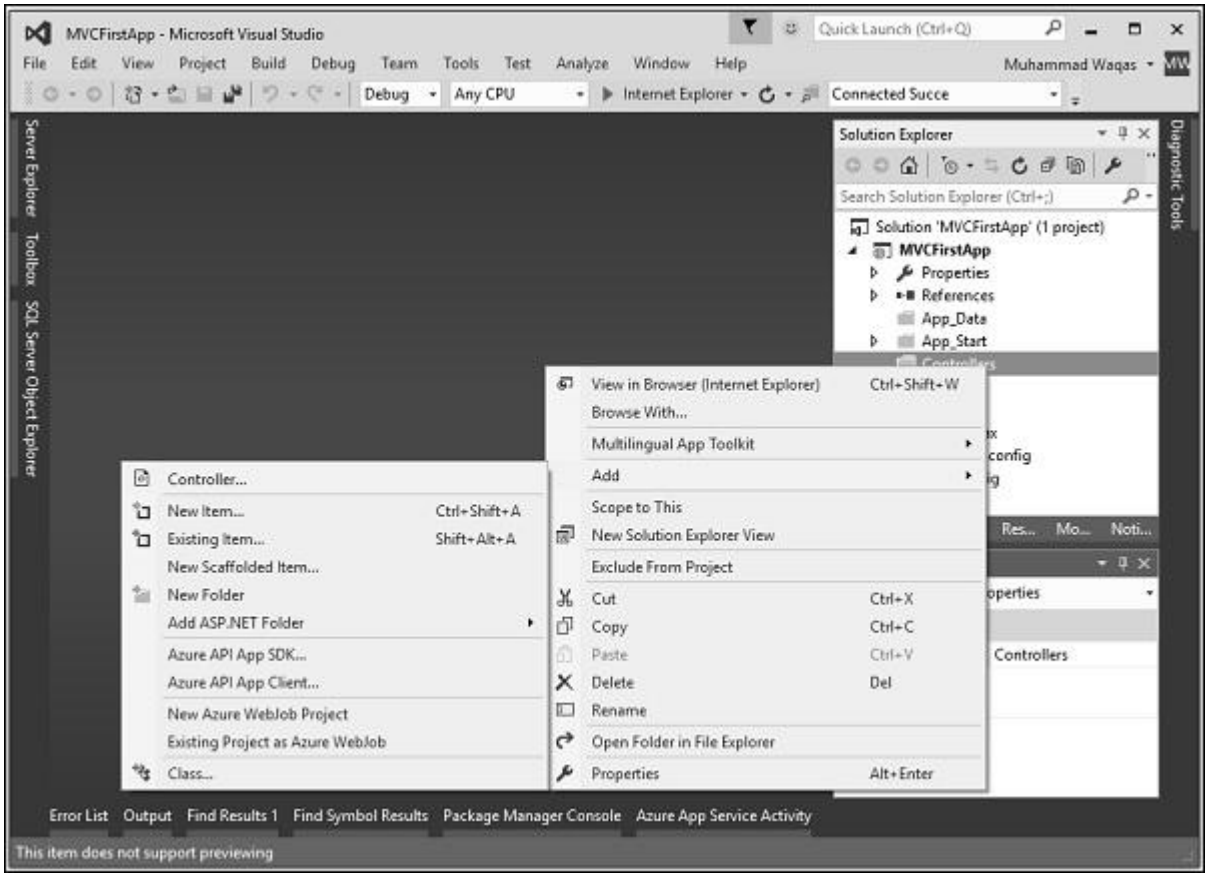


The default browser is, Internet Explorer, but you can select any browser that you have installed from the toolbar.

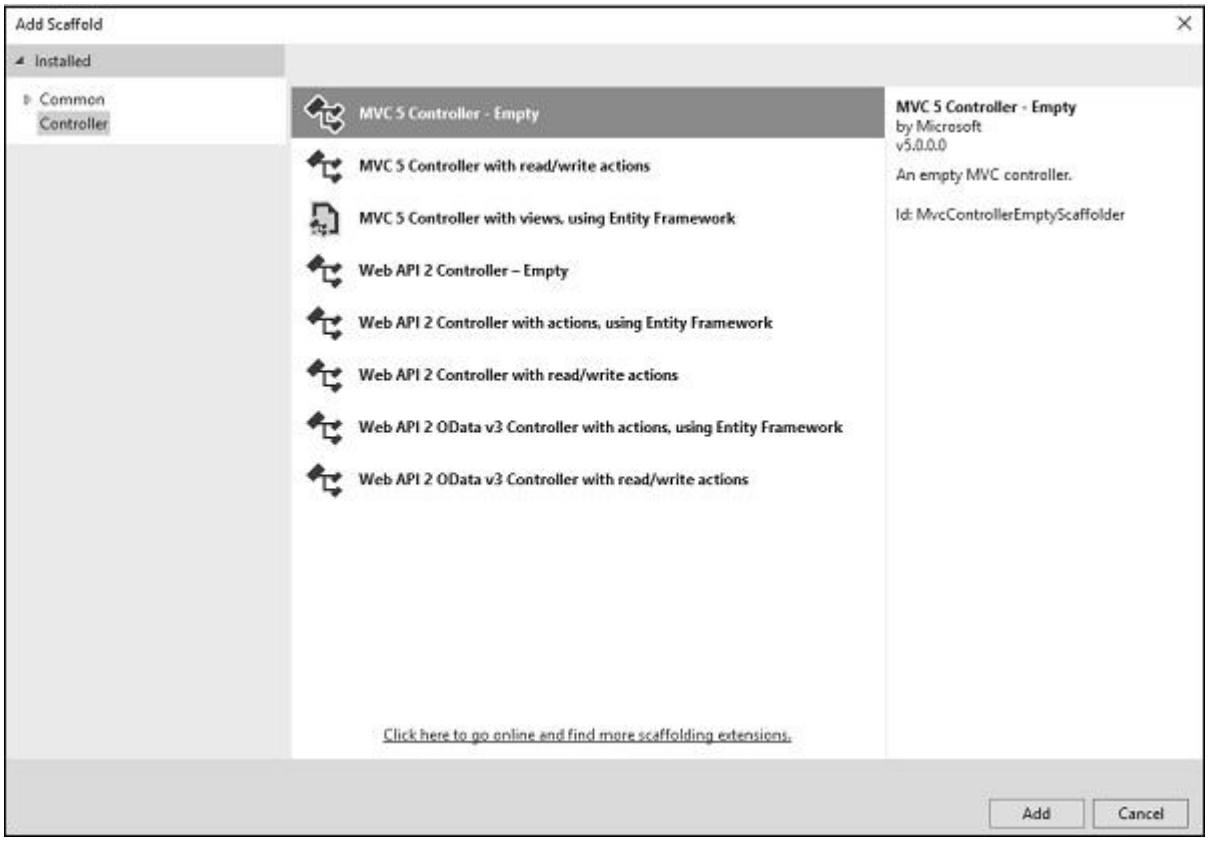
Add Controller

To remove the 404 Not Found error, we need to add a controller, which handles all the incoming requests.

Step 1 − To add a controller, right-click on the controller folder in the solution explorer and select Add → Controller.

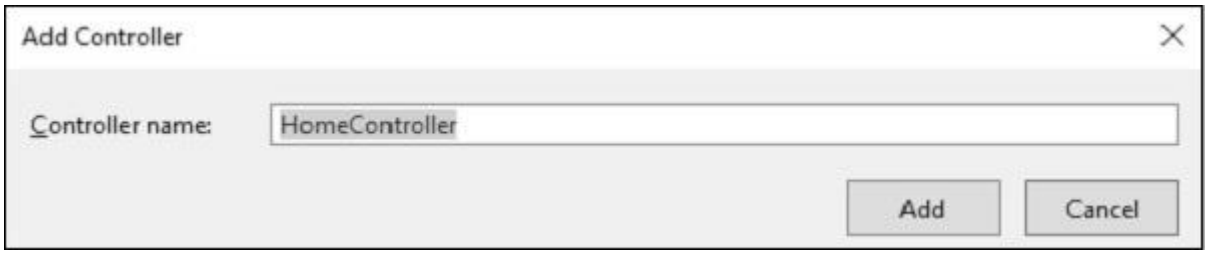


It will display the Add Scaffold dialog.



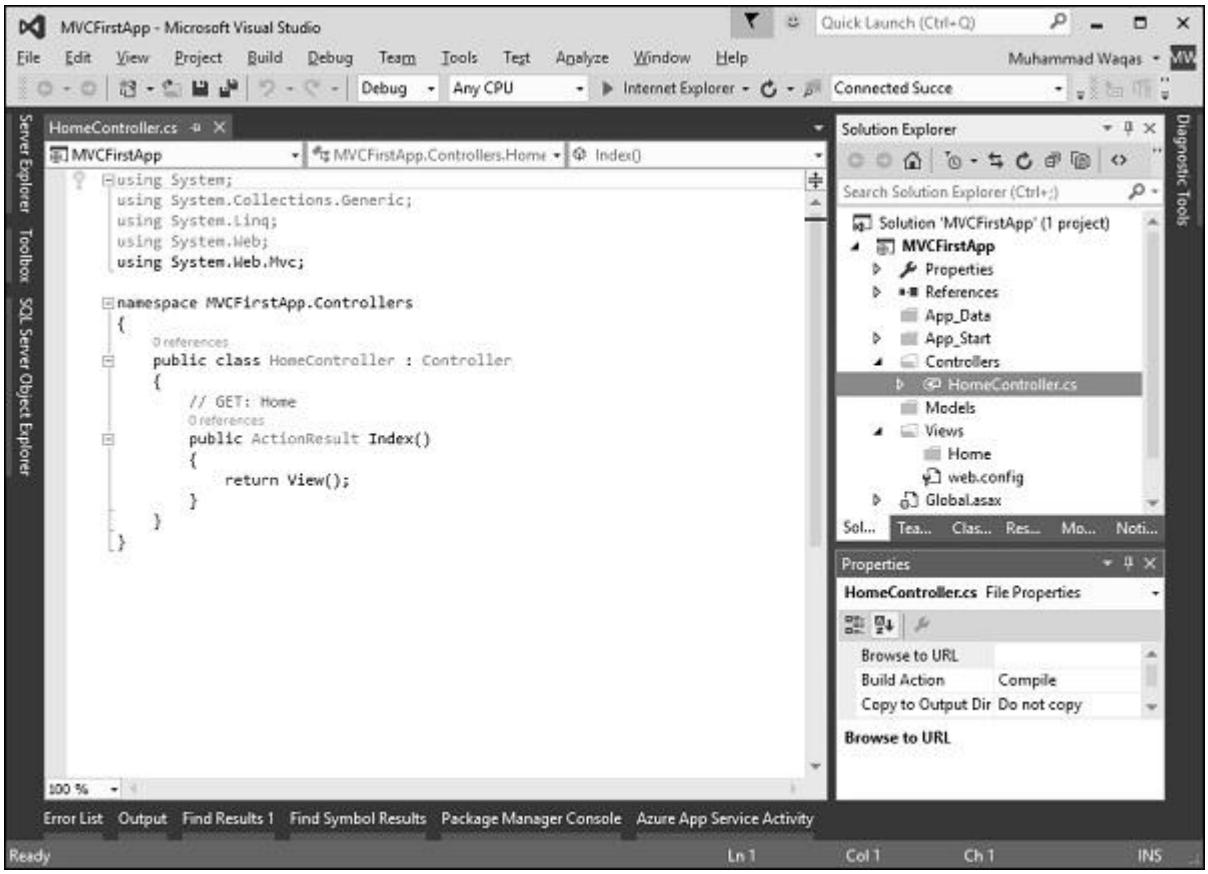
Step 2 − Select the MVC 5 Controller Empty option and click Add button.

The Add Controller dialog will appear.



Step 3 − Set the name to HomeController and click the Add button.

You will see a new C# file HomeController.cs in the Controllers folder, which is open for editing in Visual Studio as well.



Step 4 − To make this a working example, lets modify the controller class by changing the action method called **Index** using the following code.

using System;

using System.Collections.Generic;

using System.Linq;

using System.Web;

using System.Web.Mvc;

```
namespace MVCFirstApp.Controllers {
  public class HomeController : Controller {
    // GET: Home
    public string Index(){
      return "Hello World, this is ASP.Net MVC Tutorials";
    }
  }
}
```

Step 5 − Run this application and you will see that the browser is displaying the result of the Index action method.

ASP.NET CORE

ASP.NET Core is the new web framework from Microsoft. It has been redesigned from the ground up to be fast, flexible, modern, and work across different platforms. Moving forward, ASP.NET Core is the framework that can be used for web development with .NET. If you have any experience with MVC or Web API over the last few years, you will notice some familiar features. At the end this tutorial, you will have everything you need to start using ASP.NET Core and write an application that can create, edit, and view data from a database.

A Brief History of ASP.NET

ASP.NET has been used from many years to develop web applications. Since then, the framework went through a steady evolutionary change and finally led us to its most recent descendant ASP.NET Core 1.0.

- ASP.NET Core 1.0 is not a continuation of ASP.NET 4.6.
- It is a whole new framework, a side-by-side project which happily lives alongside everything else we know.
- It is an actual re-write of the current ASP.NET 4.6 framework, but much smaller and a lot more modular.
- Some people think that many things remain the same, but this is not entirely true. ASP.NET Core 1.0 is a big fundamental change to the ASP.NET landscape.

NET Core is a free open source, a general-purpose development platform for developing modern cloud-based software applications on Windows, Linux, and macOS operating systems. It operates across several platforms and has been revamped to make .NET fast, scalable, and modern. .NET Core is one of Microsoft's big contributions and released under the MIT License. It offers the following features:

- Cross-Platform
- Open Source

- High Performance
- Multiple environments and development mode etc.

Differences between .Net Core and .Net Framework:

BASED ON	.NET Core	.NET Framework
Open Source	.Net Core is an open source.	Certain components of the .Net Framework are open source.
Cross-Platform	Works on the principle of "build once, run anywhere". It is compatible with various operating systems — Windows, Linux, and Mac OS as it is cross-platform.	.NET Framework is compatible with the windows operating system. Although, it was developed to support software and applications on all operating systems.
Application Models	.Net Core does not support desktop application development and it rather focuses on the web, windows mobile, and windows store.	.Net Framework is used for the development of both desktop and web applications as well as it supports windows forms and WPF applications.
Installation	.NET Core is packaged and installed independently of the underlying operating system as it is cross-platform.	.NET Framework is installed as a single package for Windows operating system.
Support for Micro-Services and REST Services	.Net Core supports the development and implementation of micro-services and the user has to create a REST API for its implementation.	.Net Framework does not support the development and implementation of microservices but it supports the REST API services.
Performance and Scalability	.NET Core offers high performance and scalability.	.Net Framework is less effective in comparison to .Net Core in terms of performance and scalability of applications.

BASED ON	.NET Core	.NET Framework
Compatibility	.NET Core is compatible with various operating systems — Windows, Linux, and Mac OS.	.NET Framework is compatible only with the Windows operating system.
Android Development	.NET Core is compatible with open-source mobile application platforms, i.e. Xamarin, through the .NET Standard Library. Developers use Xamarin's tools to configure the mobile app for specific mobile devices such as iOS, Android, and Windows phones.	.NET Framework does not support any framework for mobile application development.
Packaging and Shipping	.Net Core is shipped as a collection of Nugget packages.	All the libraries of .Net Framework are packaged and shipped together.
Deployment Model	Whenever the updated version of .NET Core gets initiated; it is updated instantly on one machine at a time, thereby getting updated in new directories/folders in the existing application without affecting it. Thus, .NET Core has a good and flexible deployment model.	In the case of .Net Framework, when the updated version is released it is first deployed on the Internet Information Server only.
Support	It has support for microservices.	It does not support creation and microservices.
WCF Services	It has no support for WCF services.	It has excellent support for WCF services.
Rest APIs	Supports Rest APIs	It also supports REST Services.

BASED ON	.NET Core	.NET Framework
CLI Tools	.NET Core provides light-weight editors and command-line tools for all supported platforms.	.Net Framework is heavy for Command Line Interface and developers prefer to work on the lightweight Command Line Interface.
Security	.NET Core does not have features like Code Access Security.	Code access security feature is present in .NET Framework.

Web Server

System that stores and delivers web content to users over the internet is known as a web server, which primarily uses HTTP or HTTPS protocols. An HTTP server specifically focuses on handling HTTP requests and responses between the client and the server.

- Responds to browser requests by serving web resources.
- HTTP server is a type of web server focused on HTTP communication.
- Some web servers support additional protocols beyond HTTP.

Working

When a user enters a URL, the browser sends an HTTP request to the web server, which processes it and returns the required resources to display the page.

- **Client Request:** In the web browser **(https://www.geeksforgeeks.org/)**, the user enters a URL.
- **DNS Resolution:** The browser contacts a DNS server to obtain the IP address of the requested domain.
- **Establishing Connection:** Using the obtained IP address, the browser establishes a connection with the web server through TCP (and TLS in case of HTTPS).
- **Sending HTTP Request:** The browser sends an HTTP request (with headers and method like GET/POST) to the server.
- **Processing Request:** The web server receives the request, processes it, and may interact with backend services or databases.
- **Serving the Response:** The server sends back a response containing status codes and requested files (HTML, CSS, JavaScript, images).
- **Rendering the Web Page:** Based on the received data, the browser parses, executes scripts, and displays the web page to the user.

Types

Web servers can be categorized based on their functionality, usage, and implementation. Below are some of the most common types:

1. Apache Web Server

Apache Web Server is a widely used open-source web server developed by the Apache Software Foundation. Released in 1995, it is written in C, highly customizable, and distributed under the Apache License 2.0.

- Supports multiple operating systems (Windows, Linux, macOS).
- Allows advanced routing.
- Provides directory-level configuration.

2. Nginx Web Server

Nginx (pronounced "Engine-X") is a high-performance web server known for speed, scalability, and efficient handling of concurrent connections. Developed by Igor Sysoev and released in 2004, it is written in C.

- Designed to handle high traffic efficiently and serve static content.
- Functions as a reverse proxy and load balancer.

3. Microsoft IIS (Internet Information Services)

IIS is a web server developed by Microsoft, designed to work with Windows Server environments. It was developed by Microsoft, and first released in 1995 as a web server designed specifically for Windows-based systems. It is written in C++.

- Supports ASP.NET, PHP, and other web technologies.
- Provides built-in security features.
- Integrates well with Microsoft products.

Web Servers and Their Use Cases

Choosing the right web server depends on what you need for your website or application. Here's a simple guide to help you decide:

- **Apache:** Reliable and customizable for general-purpose websites.
- **Nginx:** High-performance server for heavy traffic.
- **IIS:** Best for Windows and ASP.NET applications.

Differentiate Between Apache and IIS

	Apache HTTP Server	Microsoft IIS
License	Open-source (Free)	Proprietary (Bundled with Windows)
Operating System	Linux, Unix, Windows, macOS	Windows OS Only
Best Paired Tech	PHP, Python, MySQL, Linux (LAMP)	.NET, C#, MSSQL, Active Directory
Configuration	Text files (httpd.conf, .htaccess)	Graphical User Interface (IIS Console)
Support Model	Global developer community	Official Microsoft Support channels

Sample Questions:

C# and .NET Fundamentals

1. Explain the features of C# programming language and discuss why it is considered modern, object-oriented, and type-safe.
2. Explain .Net framework. Explain features and advantages of .Net framework.
3. What is Common Language runtime (CLR)? Explain features of Common Language runtime (CLR).
4. Describe the architecture of the .NET Framework and explain the roles of CLR and FCL in detail.
5. Discuss the design principles of the .NET Framework such as interoperability, portability, security, and memory management.
6. Explain how C# supports object-oriented programming concepts with suitable examples. Describe the unified type system in C# and explain its advantages.

Object-Oriented Programming (OOP)

5. Explain the concepts of encapsulation, abstraction, inheritance, and polymorphism in C# with examples.
6. Compare Procedure-Oriented Programming (POP) and Object-Oriented Programming (OOP) in detail.
7. Explain the concept of classes and objects in C# and how they are used in application development. Discuss the importance of data hiding and information hiding in OOP.

C# Advanced Concepts

8. Explain type safety in C# and discuss its importance in secure programming. Discuss properties, methods, and events in C# with examples.
9. Describe memory management in C# and explain the role of the garbage collector.
10. Explain access modifiers in C# and their impact on program security and accessibility.

Exception Handling & Multithreading

11. Explain exception handling in C# using try, catch, finally, and throw with examples.
12. Describe different types of predefined exceptions in C# and their uses.
13. Explain the concept of multithreading in C# and its advantages.

ADO.NET

14. Compare ADO and ADO.NET in detail with suitable examples.
15. Explain the architecture of ADO.NET and its components. Describe the role of ***SqlConnection*** and ***SqlCommand*** in database operations.
16. Explain the difference between ***DataReader*** and ***DataSet*** with examples.
17. Discuss ***DataAdapter*** and ***DataTable*** and their importance in disconnected architecture.

ASP.NET Web Applications

18. Explain the architecture and components of ASP.NET web applications. Explain event handling in ASP.NET with suitable examples
19. Describe different types of ASP.NET controls with examples.
20. Discuss validation controls in ASP.NET and explain their types and uses.
21. Suppose you are hired by a reputed software company which is going to design an application for "***Employee Management System***". Your responsibility is to design a schema named EMS and create a table named Employee (EmpNo, Name, Email, phone number, Company, Department, Joining Date, and Salary). Write a program to design a ASP.NET form to take input for this table and insert the data into table after clicking the Save button.

ASP.NET MVC/Core/Web Server

22. Explain ASP.NET MVC in detail. Discuss its architecture, components (Model, View, and Controller), features, advantages, and applications with a neat diagram.
23. Compare ASP.NET Framework and ASP.NET Core.
24. Explain the concept of a Web Server. Describe the working process of a web server from the moment a user enters a URL in the browser until the requested web page is displayed.

Resources:

https://www.aspsnippets.com/Categories/C.Net/
https://www.aspsnippets.com/Categories/ASP.Net/
https://www.javatpoint.com/c-sharp-tutorial
https://www.javatpoint.com/ado-net-tutorial
https://www.javatpoint.com/asp-net-tutorial

Sample Project:
https://www.sourcecodester.com/